

The mathematics of YeeLab

Voter models, exact ranking probabilities, voting methods, and how Beta and normal voters differ

This document explains what YeeLab computes and why. Chapter 1 defines the model: the Yee diagram, the two voter distributions (normal and Beta) and the spread rules that fix the Beta distributions. Chapter 2 shows how the share of every ranking of the candidates is computed exactly for every pixel, and Chapter 3 how the voting methods turn these shares into winners. Chapter 4 explains why Beta and normal voters give different diagrams. Chapter 5 compares the spread rules and explains why *rms* is the default. Chapter 6 asks when a candidate's region is convex or in one piece, and how that relates to the monotonicity of the method. Chapter 7 shows how the web UI draws the regions as curves fast enough to follow a dragged candidate.

The settings that appear in the mathematics, with their defaults (file names are in the package `src/yeeLab/`):

quantity	symbol	set by	default
pixels per axis	n	--pixels (<code>pixels/plot.py</code>), <code>PIXELS</code> (<code>shares.py</code>)	400 / 300
deviation	D	--deviation, <i>Deviation</i> slider	0.2
voter distribution		--distribution, <i>Voters</i>	Beta
Beta spread rule		--spread, <i>Beta spread fixed</i>	<i>rms</i>
interpolation nodes per axis	N	--nodes (<code>NODES</code>)	49
Gauss-Legendre points per edge	Q	<code>QUAD_NODES</code>	24
exponent of tapered	τ	<code>TAPER</code>	0.2
contour grid per axis (UI)	G	<code>DRAG_GRID</code> / <code>FINAL_GRID</code> (<code>web/app.py</code>)	160 / 320

Contents

1. The model	2
1.1. Yee diagrams	2
1.2. Normal voters	2
1.3. Beta voters	3
1.4. The deviation and the centre pixel	3
1.5. Spread rules	3
1.6. Solving for the parameters (<code>beta_params_at</code>)	5
2. Exact ranking probabilities	7
2.1. Ranking cells (<code>ranking_cells</code>)	7
2.2. Cell probability as a boundary integral	8
2.3. Edge integrals (<code>_edge_integral</code>)	9
2.4. Interpolation in the pixel median	10
2.5. Normal voters (<code>normal.py</code>)	10
2.6. Accuracy and tests	12
3. Voting methods (<code>pixels/methods.py</code>)	13
3.1. First past the post (<code>fptp</code>)	13
3.2. Instant runoff (<code>irv</code>)	13
3.3. Borda count (<code>borda</code>)	13
3.4. Pairwise majorities	13
3.5. Ties	13
4. Beta versus normal voters	14
4.1. The two models	14
4.2. Pairwise majorities	15
4.3. Curved borders	16

4.4. Summary	19
5. Choosing the spread rule	20
5.1. The three rules on the default candidates	20
5.2. The round IRV edge of <code>mean_abs</code>	21
5.3. What keeps a border straight	22
5.4. Benchmark	22
5.5. Other ways to fix the spread	23
5.6. Conclusion	24
6. Shapes of win regions	25
6.1. Regions, shapes and monotonicity	25
6.2. Why monotonicity does not fix the shape	25
6.3. Where the regions are convex	25
6.4. Monotone methods with non-convex regions	26
6.5. Monotone methods with regions in pieces	28
6.6. Condorcet methods under Beta voters	28
6.7. Summary	29
7. Real-time diagrams in the web UI	30
7.1. What each method needs (<code>shares.py</code>)	30
7.2. Dragging: an edge cache	30
7.3. Tabulated Beta CDF (<code>beta_tables.py</code>)	30
7.4. Compiled kernels (<code>numba</code>)	31
7.5. Borders as zero sets (<code>methods.py</code> , <code>regions.py</code>)	32
7.6. Timings	33

1. The model

1.1. Yee diagrams

The political space is the unit square $[0, 1]^2$. There are C candidates at fixed points $c_1, \dots, c_C \in [0, 1]^2$. The square is divided into $n \times n$ pixels, and every pixel is its own election:

- the voters of the pixel are spread around the pixel centre $m = ((k + \frac{1}{2})/n, (l + \frac{1}{2})/n)$, $k, l = 0, \dots, n - 1$;
- every voter ranks the candidates by Euclidean distance, closest first;
- the pixel is coloured by the winner of a voting method.

A pixel does not have a finite number of voters. Its electorate is a probability distribution, and all that a voting method needs from it is, for every strict ranking r of the candidates,

$$P_{kl}(r) = \text{share of the voters of pixel } (k, l) \text{ whose ranking is } r. \quad (1)$$

Chapter 2 computes these shares exactly, without sampling voters, so the diagrams have no Monte Carlo noise. A voter can only be tied between two candidates on a line (Section 2.1), and lines have probability zero, so ties are ignored.

If every voter sat exactly at the pixel centre, every method would elect the candidate nearest to m , and the diagram would be the **Voronoi diagram** of the candidates (`pixels.methods.voronoi`, and deviation 0 in the web UI). It is the reference against which the other diagrams are compared.

1.2. Normal voters

In the original Yee diagrams the voters of a pixel are isotropic normal around the pixel centre,

$$(X, Y) \sim \mathcal{N}(m, \sigma^2 I), \quad (2)$$

so some of them lie outside the square. The spread is set by the **deviation** D , the mean absolute deviation of each coordinate from the pixel centre. For $X \sim \mathcal{N}(m_x, \sigma^2)$, $E|X - m_x| = \sigma\sqrt{2/\pi}$, so

$$\sigma = D\sqrt{\frac{\pi}{2}} \quad (D = 0.2 \Rightarrow \sigma = 0.251, \quad D = 0.3 \Rightarrow \sigma = 0.376) \quad (3)$$

(normal.sigma_from_deviation).

1.3. Beta voters

YeeLab's own model keeps every voter inside the square. The two coordinates are independent and Beta distributed,

$$X \sim \text{Beta}(a_x, b_x), \quad Y \sim \text{Beta}(a_y, b_y), \quad (4)$$

where (a_x, b_x) depend only on the pixel column and (a_y, b_y) only on the pixel row. Both come from the same function of the median, so it is enough to study one coordinate $X \sim \text{Beta}(a, b)$ with median $m \in (0, 1)$. We write f, F for the pdf and CDF of X and g, G for those of Y . The CDF of $\text{Beta}(a, b)$ is the regularised incomplete beta function $I_x(a, b)$ (scipy.special.betainc), its inverse is betaincinv.

A Beta distribution has two parameters, so two conditions fix it:

1. the **median** is the pixel centre,

$$I_m(a, b) = \frac{1}{2}; \quad (5)$$

2. a **spread rule** fixes how far the voters spread (Section 1.4, Section 1.5).

The median plays the role that the mean plays for normal voters: a line through m parallel to an axis has exactly half of the pixel's voters on each side. For oblique lines this is no longer true, and Chapter 4 shows that this is the root of most differences between the two models.

If $X \sim \text{Beta}(a, b)$ then $1 - X \sim \text{Beta}(b, a)$. So the pixel with median $1 - m$ has the parameters of median m swapped, and the rules below only need to be defined for $m \geq \frac{1}{2}$.

1.4. The deviation and the centre pixel

At the centre pixel, $m = \frac{1}{2}$, symmetry forces $a = b = a_0$, and every spread rule fixes a_0 by the same condition as the normal model, $E|X - \frac{1}{2}| = D$. For a symmetric Beta this has a closed form,

$$E|X - \frac{1}{2}| = \frac{2^{-2a_0}}{a_0 B(a_0, a_0)} = D, \quad (6)$$

which decreases from $\frac{1}{2}$ ($a_0 \rightarrow 0$) to 0 ($a_0 \rightarrow \infty$), so $0 < D < \frac{1}{2}$ is required. It is solved for a_0 by bracketing (centre_shape). The uniform distribution, $a_0 = 1$, has $D = \frac{1}{4}$ and separates two shapes:

- $D < \frac{1}{4}$ gives $a_0 > 1$: the voters of the centre pixel are bell-shaped, dense in the middle and thin at the walls. The default $D = 0.2$ gives $a_0 = 1.724$.
- $D > \frac{1}{4}$ gives $a_0 < 1$: the density is U-shaped and infinite at both walls, with more voters near the walls than in the middle. $D = 0.3$ gives $a_0 = 0.602$ ($\text{Beta}(\frac{1}{2}, \frac{1}{2})$ has $D = 1/\pi \approx 0.318$).

Away from the centre, as the median approaches a wall, one of the parameters drops below 1 for every D (at $D = 0.2$ with rms, $b = 0.42$ at $m = 0.9$), and the density becomes infinite at that wall. Much of the numerics of Chapter 2 is shaped by this.

1.5. Spread rules

1.5.1. One free parameter

With the median fixed by Equation 5, one parameter is left. The convenient one is the **concentration** $\kappa = a + b$:

$$\text{Var } X = \frac{\mu(1-\mu)}{\kappa + 1}, \quad \mu = EX = \frac{a}{\kappa}. \quad (7)$$

For given κ and m , Equation 5 determines a and b (Section 1.6). A spread rule is therefore a curve $\kappa(m)$ for $m \in [\frac{1}{2}, 1)$ that starts at $\kappa(\frac{1}{2}) = 2a_0$; a larger κ means voters closer together.

1.5.2. Why the spread must change near a wall

Near a wall, “the same spread for every pixel” cannot hold in every sense. When the median m is close to 1, the half of the voters above the median lies within $1 - m$ of it and contributes almost nothing to any measure of spread; the whole spread has to come from the other half, the one towards the centre of the square. For the mean absolute deviation,

$$E|X - m| = \frac{1}{2}E[m - X \mid X < m] + \frac{1}{2}E[X - m \mid X > m] \leq \frac{1}{2}E[m - X \mid X < m] + \frac{1 - m}{2}, \quad (8)$$

so keeping $E|X - m| = D$ forces

$$E[m - X \mid X < m] \geq 2D - (1 - m). \quad (9)$$

At the centre this far half lies at mean distance D from the median; next to a wall it must lie twice as far. For the power mean $(E|X - m|^p)^{\frac{1}{p}}$ the same argument gives a factor $2^{\frac{1}{p}}$: 2 for $p = 1$, $\sqrt{2} \approx 1.41$ for the root mean square, 1.26 for $p = 3$. This holds for every distribution on $[0, 1]$, not only for Beta. Every rule therefore decides, in effect, how far the far half of the voters is pushed out near a wall, and this is what the three rules differ in.

1.5.3. The three rules

rms (default). The root mean square distance of the voters from the median is the same as at the centre pixel,

$$\sqrt{E(X - m)^2} = s = \frac{1}{2\sqrt{2a_0 + 1}}, \quad E(X - m)^2 = \text{Var } X + (\mu - m)^2, \quad (10)$$

where s is the standard deviation of $\text{Beta}(a_0, a_0)$ ($s = 0.237$ for $D = 0.2$, 0.337 for $D = 0.3$). The square weights distant voters more, so near a wall a few voters far out make up the spread and the rest of the far half stays where it is. It is the rule with the clearest meaning, and its borders bend only slightly more than those of the best rule found (Section 5.4).

tapered. The concentration is prescribed directly,

$$\kappa(m) = 2a_0(4m(1 - m))^\tau, \quad \tau = 0.2. \quad (11)$$

Since $\mu(1 - \mu)$ is the largest variance any distribution on $[0, 1]$ with mean μ can have, $1/(\kappa + 1)$ is the share of that room which the electorate of the pixel uses, and $4m(1 - m)$ measures the room around the median (1 at the centre, 0 at a wall). The rule lets electorates near a wall use a little more of their room; at $D = 0.3$: 45 % at the centre, 50 % at $m = 0.9$, 58 % at $m = 0.98$. The exponent is empirical: it gave the straightest Condorcet borders at equal numbers of cycles in a benchmark (Section 5.4). The rule behaves very much like rms. Only within about 10^{-5} of a wall, which no practical pixel grid reaches, does κ become so small that the spread grows again.

mean_abs (legacy). The mean absolute deviation is D for every median, $E|X - m| = D$. This was the first rule, and it is kept to reproduce older results. Because it keeps the mean absolute deviation fixed, Equation 9 applies in full: near a wall the far half of the voters is pushed twice as far out, towards the opposite wall. At median 0.98 the share of voters below 0.1 is back at 0.192, more than at the centre, where rms has 0.049 (Figure 1). This push bends Condorcet borders, adds cycles and gives IRV a round edge that the other rules do not have (Chapter 5).

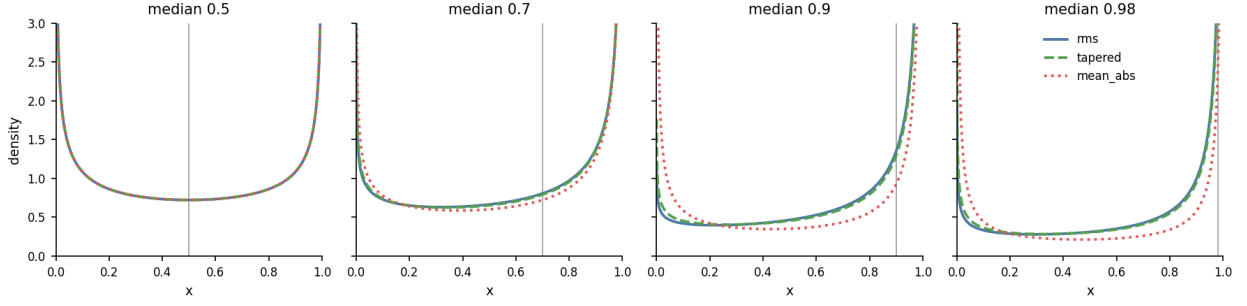


Figure 1: Density of one coordinate of a pixel's voters, $D = 0.3$, for medians moving towards the wall at 1 (grey: the median). All rules agree at the centre. Near the wall mean_abs moves many voters to the opposite wall; rms and tapered hardly differ.

At $D = 0.3$ (docs/figures.py):

median	rule	$\kappa = a + b$	mean	$E X - m $	$\sqrt{E(X - m)^2}$	$P(X < 0.1)$
0.50	all	1.204	0.500	0.300	0.337	0.175
0.70	rms	1.197	0.621	0.283	0.337	0.095
	tapered	1.163	0.619	0.286	0.340	0.099
	mean_abs	1.012	0.612	0.300	0.355	0.121
0.90	rms	1.062	0.750	0.229	0.337	0.051
	tapered	0.982	0.743	0.237	0.348	0.059
	mean_abs	0.554	0.685	0.300	0.430	0.146
0.98	rms	0.799	0.812	0.186	0.337	0.049
	tapered	0.724	0.801	0.197	0.353	0.059
	mean_abs	0.307	0.698	0.300	0.490	0.192

1.6. Solving for the parameters (beta_params_at)

Only the distinct values of $\max(m, 1 - m) \geq \frac{1}{2}$ are solved, in increasing order; medians below $\frac{1}{2}$ get the mirrored parameters.

The median for a given concentration (`_b_for_median`). For a fixed $\kappa = a + b$ the median of $\text{Beta}(\kappa - b, b)$ decreases from 1 to $\frac{1}{2}$ as b grows from 0 to $\kappa/2$, so Equation 5 has exactly one root, found by bracketing b (Brent's method). tapered computes $\kappa(m)$ from Equation 11 and needs nothing else.

rms (`_solve_rms`). Along the Beta distributions with median m , the RMS distance Equation 10, which is explicit in (a, b) , decreases as κ grows. An outer bracketing search over $\log \kappa$, starting from the solution at the previous median, finds the κ whose RMS distance is s .

mean_abs (`_solve_beta`). Split the absolute deviation at the median and use $F(m) = \frac{1}{2}$:

$$\begin{aligned}
 E|X - m| &= E[X - m] - 2E[(X - m)\mathbb{1}\{X < m\}] \\
 &= \mu - m - 2E[X\mathbb{1}\{X < m\}] + 2mF(m) \\
 &= \mu - 2E[X\mathbb{1}\{X < m\}].
 \end{aligned} \tag{12}$$

Since $x \cdot x^{a-1}(1-x)^{b-1}/B(a, b) = \mu \cdot$ (density of $\text{Beta}(a+1, b)$), $E[X\mathbb{1}\{X < m\}] = \mu I_m(a+1, b)$, and therefore

$$E|X - m| = \frac{a}{a+b}(1 - 2I_m(a+1, b)) = D. \tag{13}$$

Equation 5 and Equation 13 are solved together for (a, b) with Levenberg-Marquardt (`scipy.optimize.root(method="lm")`). A two-dimensional solver needs a good initial guess, so the medians

are solved from the centre solution (a_0, a_0) outwards, each solution being the initial guess for the next median, and intermediate medians are inserted wherever two consecutive medians are more than 0.25 apart in $\text{logit}(m)$ (CONTINUATION_STEP). This keeps the solver on the correct branch, also for a single median close to 0 or 1.

The tests check that every rule keeps its quantity fixed (to 10^{-9} , with the moments computed by independent quadrature), the median (to 10^{-12}), the mirror symmetry, and that a lone median gets the same parameters as a full sweep.

2. Exact ranking probabilities

This chapter computes the shares (Equation 1) for Beta voters (`ranking_cells.py` and `pixels/beta.py`, Section 2.1 to Section 2.4) and for normal voters (`normal.py` and `pixels/normal.py`, Section 2.5). The only approximations are Gauss–Legendre quadrature and polynomial interpolation, both of which converge exponentially fast; the normal model needs no quadrature at all. For Beta voters the steps are:

1. Solve the Beta parameters at the interpolation nodes (Section 1.6, Section 2.4).
2. Cut the square by the $\binom{C}{2}$ bisectors into cells, each with one ranking (Section 2.1).
3. Write the probability of each cell as a sum of integrals over its edges; each edge is shared by two cells (Section 2.2).
4. Compute every edge integral for all pairs of node parameters (Section 2.3).
5. Interpolate from the $N \times N$ nodes to the $n \times n$ pixels (Section 2.4).

2.1. Ranking cells (`ranking_cells`)

2.1.1. Bisectors

A voter at p prefers c_i to c_j exactly when

$$|p - c_i|^2 < |p - c_j|^2 \iff 2(c_j - c_i) \cdot p < |c_j|^2 - |c_i|^2. \quad (14)$$

The boundary of Equation 14 is the perpendicular bisector of $c_i c_j$, a straight line. A voter’s full ranking is determined by the signs of all $\binom{C}{2}$ comparisons, so the ranking can only change when the voter crosses one of these bisectors.

The $\binom{C}{2}$ lines cut the unit square into convex polygons (“cells”). Inside a cell every comparison has a fixed sign, hence **every point of a cell has the same ranking**. Points on a bisector have a tie, but lines have zero area and so zero probability.

The crucial observation is that **the cells do not depend on the pixel**: they depend only on the candidates. Only the probability mass that each pixel’s distribution puts on each cell changes. So

$$P_{kl}(r) = P_{kl}((X, Y) \in K_r) = \iint_{K_r} f_k(x)g_l(y) \, dx \, dy, \quad (15)$$

where K_r is the cell with ranking r .

2.1.2. Construction

The construction starts with the square as a single polygon and, for every pair (i, j) , splits every current polygon into its parts on either side of the bisector (half-plane clipping, `_clip`, a Sutherland–Hodgman step for one line). Slivers with area below 10^{-14} are discarded. All polygons stay convex and counter-clockwise (CCW). The ranking of a cell is found by sorting the candidates by distance from the cell’s vertex average, which lies strictly inside the convex cell.

Figure 2 shows the arrangement for the candidates A–E used throughout this document.

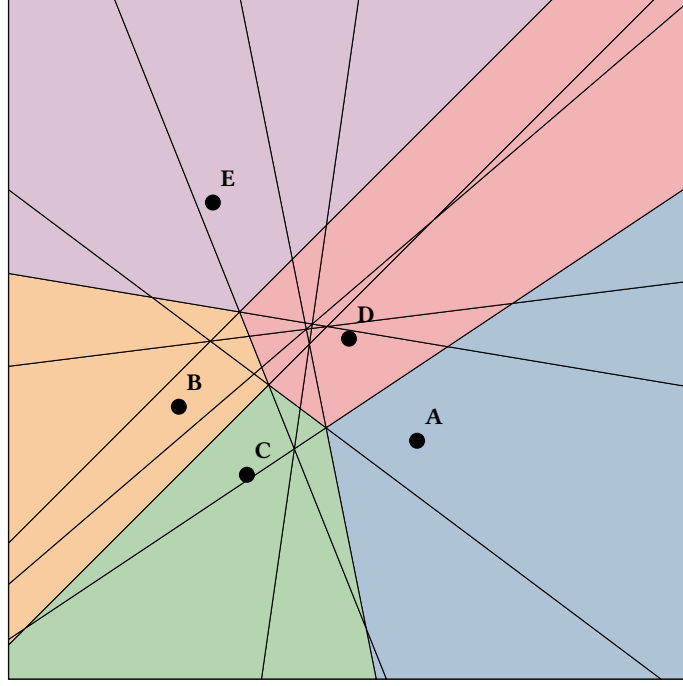


Figure 2: The 43 ranking cells of the default candidates A–E (rows of CANDIDATES). Colour marks the first choice; every cell has its own full ranking.

2.2. Cell probability as a boundary integral

Computing the double integral Equation 15 directly for every cell and every pixel would be expensive. Green's theorem turns it into a sum of integrals along the cell's edges, and each edge is shared by two cells.

2.2.1. Green's theorem

For a region K with CCW boundary ∂K and a 1-form $\omega = L dx + M dy$,

$$\oint_{\partial K} L dx + M dy = \iint_K (\partial_x M - \partial_y L) dx dy. \quad (16)$$

Choose

$$\omega = -f(x)G(y) dx. \quad (17)$$

Then $\partial_x M - \partial_y L = f(x)g(y)$, which is the joint density, so

$$P(K) = \oint_{\partial K} \omega = \sum_{\text{edges } s \rightarrow e} \int_s^e \omega. \quad (18)$$

An equally valid choice is

$$\omega' = F(x)g(y) dy, \quad \omega' - \omega = Fg dy + fG dx = d(FG). \quad (19)$$

Because ω' and ω differ by an exact form, on any segment

$$\int_s^e \omega = \int_s^e \omega' - [F(x)G(y)]_s^e. \quad (20)$$

Which form is integrated numerically is chosen edge by edge (Section 2.3).

Sanity check. The probabilities of all cells sum to one: interior edges are traversed once in each direction and cancel, leaving the boundary of the square. On it $dx = 0$ on the vertical sides, $G(0) = 0$ on the bottom, and the top ($y = 1$, traversed from $x = 1$ to $x = 0$) gives $-\int_1^0 f(x) dx = 1$.

2.2.2. Sharing edges (compute_ranking_probabilities)

Every edge is stored once under a canonical orientation (lexicographically smaller endpoint first). A cell that traverses the edge in the canonical direction adds its integral with sign +1; the neighbouring cell traverses it the other way and adds it with sign -1 . The edges are integrated in parallel threads (betainc / betaincv release the GIL).

The result of each edge integral is a matrix: entry $[k, l]$ uses the X -parameters of pixel column k and the Y -parameters of pixel row l . Because X and Y are independent, one set of n parameter pairs serves both axes, and the whole $n \times n$ grid of pixels comes out of $O(n)$ one-dimensional quantities combined in an outer-product fashion.

2.3. Edge integrals (_edge_integral)

Let the edge go from $s = (x_s, y_s)$ to $e = (x_e, y_e)$.

2.3.1. Axis-parallel edges (closed form)

- **Vertical** ($x_s = x_e$): $dx = 0$, so $\int \omega = 0$.
- **Horizontal** ($y_s = y_e$): $G(y)$ is constant, so

$$\int_s^e \omega = -G(y_s)(F(x_e) - F(x_s)). \quad (21)$$

These include all edges on the boundary of the square.

2.3.2. General edges: substitution $u = F(x)$

On the line through s and e , $y = \ell(x) = y_s + (x - x_s)(y_e - y_s)/(x_e - x_s)$. Integrating Equation 17 directly is a bad idea: $f(x)$ is infinite at 0 or 1 when $a < 1$ or $b < 1$ (Section 1.4), and quadrature would converge slowly. Substituting $u = F(x)$, $du = f(x) dx$ absorbs the density:

$$\int_s^e \omega = - \int_{F(x_s)}^{F(x_e)} G(\ell(F^{-1}(u))) du. \quad (22)$$

The integrand is now bounded by 1 and smooth unless the edge reaches $y = 0$ or $y = 1$, where G itself behaves like y^a or $1 - (1 - y)^b$ and is not smooth.

2.3.3. Edges touching $y = 0$ or $y = 1$: substitution $v = G(y)$

For those edges the roles of the axes are swapped. Integrate ω' from Equation 19 with $v = G(y)$, $dv = g(y) dy$, and $x = \ell^{-1}(y)$:

$$\int_s^e \omega' = \int_{G(y_s)}^{G(y_e)} F(\ell^{-1}(G^{-1}(v))) dv, \quad (23)$$

and convert back to ω with Equation 20. Now the endpoint on $y \in \{0, 1\}$ is harmless because it is mapped to $v \in \{0, 1\}$ and the singular behaviour of G is absorbed by the substitution, while x stays in the interior of $(0, 1)$ so F is smooth there.

2.3.4. Edges touching both kinds of boundary

An edge can run from a vertical side of the square ($x \in \{0, 1\}$) to a horizontal one ($y \in \{0, 1\}$). Neither substitution handles both endpoints, so the edge is split at its midpoint; each half touches only one kind of boundary and is handled as above. (An edge ending exactly at a corner of the square is integrated with the v form.)

2.3.5. Quadrature

The remaining one-dimensional integrals over $[u_s, u_e]$ (or $[v_s, v_e]$) are computed with Q -point Gauss–Legendre quadrature (QUAD_NODES, default $Q = 24$):

$$\int_{u_s}^{u_e} h(u) du \approx \frac{u_e - u_s}{2} \sum_{q=1}^Q w_q h\left(u_s + (u_e - u_s) \frac{t_q + 1}{2}\right), \quad (24)$$

with Legendre nodes t_q and weights w_q on $[-1, 1]$. Note that the interval $[F_k(x_s), F_k(x_e)]$ depends on the pixel column k , so the nodes are placed per pixel.

Cost. For the u form, F_k^{-1} is evaluated at Q nodes for each of the n column-parameters (Qn calls of `betaincinv`), then G_l is evaluated at each resulting y for each of the n row-parameters (Qn^2 calls of `betainc`). An edge therefore costs $O(Qn^2)$ Beta CDF evaluations, and the total cost is $O(E \cdot Q \cdot n^2)$ for E distinct edges.

2.4. Interpolation in the pixel median

With n large, $O(Qn^2)$ per edge is still costly. But the cell probabilities are **smooth** functions of the medians (m_x, m_y) : (a, b) depends smoothly on m , and the integrals Equation 15 depend smoothly on (a, b) . (The **winner** of a voting method jumps between pixels, but the probabilities do not.) So they are computed exactly only on an $N \times N$ grid of medians (NODES, default $N = 49$) and interpolated to all $n \times n$ pixels. If $N \geq n$ no interpolation is used.

2.4.1. Nodes (`node_medians`)

The interpolation variable is $z = \text{logit}(m) = \log(m/(1 - m))$. It stretches the regions near 0 and 1, where (a, b) change fastest. The nodes are the Chebyshev–Lobatto points in z spanning the first and last pixel median:

$$t_j = -\cos\left(\pi \frac{j}{N-1}\right), \quad z_j = Zt_j, \quad m_j = \text{expit}(z_j), \quad j = 0, \dots, N-1, \quad (25)$$

where $Z = \text{logit}(1 - \frac{1}{2n})$. (The code writes t_j as $\sin(\pi \frac{2j-(N-1)}{2(N-1)})$, which is the same thing.) Chebyshev–Lobatto points cluster at the ends of the interval, which further concentrates nodes near the walls and avoids the Runge phenomenon.

2.4.2. Barycentric interpolation (`_interpolation_matrix`)

For Chebyshev–Lobatto nodes the barycentric formula of the second kind is

$$p(z) = \frac{\sum_j \frac{\lambda_j}{z - z_j} h_j}{\sum_j \frac{\lambda_j}{z - z_j}}, \quad \lambda_j = (-1)^j, \quad \lambda_0, \lambda_{N-1} \text{ halved}, \quad (26)$$

where h_j are the values at the nodes. (The weights are invariant under the affine map $t \rightarrow Zt$, up to a common factor that cancels.) When a target coincides with a node, the node value is used directly. Evaluated at all n pixel medians this is a fixed $n \times N$ matrix L , and on the 2-D tensor grid

$$P_{\text{pixels}}[\cdot, \cdot, r] = L P_{\text{nodes}}[\cdot, \cdot, r] L^T, \quad (27)$$

applied as two matrix products (`interpolate_to_pixels`). The final product is carried out in `float32` because the result is by far the largest array; the rounding ($\approx 10^{-7}$) can only affect pixels whose winning margin is equally small. Values are finally clipped to $[0, 1]$.

Because the probabilities are analytic in z , the interpolation error decreases exponentially in N . The interpolation reproduces constants exactly, so the probabilities of each pixel still sum to one up to rounding. Narrow voters change faster with the median: for Beta voters with $D < 0.1$ the web UI uses $2N - 1 = 97$ nodes, because with 49 the shares of a pixel at $D = 0.05$ would sum to up to 1.025.

Only the node probabilities are cached on disk (`cache/beta/<spread>/`, `cache/normal/`), together with the medians, the parameters and the settings they were made with; interpolation happens on every load.

2.5. Normal voters (`normal.py`)

`normal.py` and `pixels/normal.py` compute the same shares for normal voters (Section 1.2), with the same interface as for Beta voters.

2.5.1. Cells in a larger box (normal_cells)

Voters can lie outside the square, so the bisector arrangement is built in the box $[-L, 1 + L]^2$ with $L = 10\sigma$ (BOX). For every pixel centre in $[0, 1]^2$ the mass outside the box is at most $4\Phi(-10) \approx 3 \cdot 10^{-23}$. The similarity map $p \mapsto (p + L)/(1 + 2L)$ maps perpendicular bisectors to perpendicular bisectors and preserves the order of distances, so `ranking_cells` is applied to the mapped candidates and the polygons are mapped back.

2.5.2. Exact cell probabilities with Owen's T function

No quadrature is needed. For a convex CCW polygon K with vertices $p_1, \dots, p_r$ and any point m , the indicator of K is, almost everywhere, the signed sum of the indicators of the triangles (m, p_k, p_{k+1}) (a fan from m ; triangles oriented clockwise count negatively). Hence

$$P(K) = \sum_{\text{edges } s \rightarrow e} P_{\pm}(m, s, e). \quad (28)$$

Each triangle (m, s, e) is split at the foot q of the perpendicular from m onto the edge line into two right triangles with legs ρ (from m to q) and t (from q along the line). The isotropic normal is invariant under rotations, so in units of σ a right triangle is $\{0 \leq z_1 \leq h, 0 \leq z_2 \leq az_1\}$ with $h = \rho/\sigma$, $a = t/\rho$, for standard normal (Z_1, Z_2) . The wedge $\{z_1 > 0, 0 < z_2 < az_1\}$ holds $\arctan(a)/(2\pi)$ of the mass, and the part of it beyond $z_1 = h$ is by definition Owen's T function,

$$T(h, a) = \frac{1}{2\pi} \int_0^a \frac{\exp(-h^2(1+x^2)/2)}{1+x^2} dx = P(Z_1 > h, 0 < Z_2 < aZ_1) \quad (h, a \geq 0). \quad (29)$$

Therefore the right triangle has probability

$$R(\rho, t) = \frac{\arctan(t/\rho)}{2\pi} - T(\rho/\sigma, t/\rho). \quad (30)$$

Both terms are odd in t ; T is even in h and odd in a , so Equation 30 with a **signed** ρ is the signed probability. For the edge $s \rightarrow e$ with unit direction u and left normal $u^\perp = (-u_y, u_x)$ let

$$\rho = (m - s) \cdot u^\perp, \quad t_s = (s - m) \cdot u, \quad t_e = t_s + |e - s|, \quad (31)$$

where $\rho > 0$ when m lies left of the edge, i.e. inside a CCW cell. Then

$$P_{\pm}(m, s, e) = R(\rho, t_e) - R(\rho, t_s), \quad (32)$$

and 0 when $\rho = 0$ (the triangle degenerates). For the $N \times N$ grid of pixel centres ρ, t_s, t_e are outer sums, so each edge costs two evaluations of Owen's T per pixel. They are compiled (Section 7.4): for $0 \leq a \leq 1$ the integral above with 12-point Gauss–Legendre, which agrees with `scipy's` `owens_t` to 10^{-16} for every h (the integrand is analytic in x), and for $a > 1$ the identity

$$T(h, a) + T(ah, 1/a) = \frac{1}{2}(\Phi(h) + \Phi(ah)) - \Phi(h)\Phi(ah) \quad (h, a \geq 0), \quad (33)$$

which leaves $T(ah, 1/a)$ with $1/a < 1$. Five candidates take about 0.02 s on 49×49 nodes.

2.5.3. Nodes

Unlike the Beta parameters, nothing changes faster near 0 or 1, so the nodes are Chebyshev–Lobatto points in the median itself, $m_j = \frac{1}{2} + (\frac{1}{2} - \frac{1}{2n})t_j$, and the barycentric interpolation of Section 2.4 is applied in m instead of `logit(m)` (`interpolate_to_pixels(..., transform=...)`). The logit nodes would be far too sparse in the middle for small σ (maximum error at 120 pixels, $N = 49$):

nodes in	$\sigma = 0.376$	$\sigma = 0.125$	$\sigma = 0.063$
m	$1.7 \cdot 10^{-7}$	$3.6 \cdot 10^{-7}$	$2.9 \cdot 10^{-7}$
<code>logit(m)</code>	$1.6 \cdot 10^{-7}$	$7.5 \cdot 10^{-5}$	$1.4 \cdot 10^{-2}$

The floor of about 10^{-7} is the `float32` rounding of the result.

2.6. Accuracy and tests

The tests (tests/) check the probabilities against quantities that are known independently:

- the cells partition the square (or the box) and every point inside a cell has the cell's ranking;
- the shares of every pixel sum to one (to 10^{-12} before interpolation);
- for a bisector parallel to an axis the share preferring one candidate is a Beta CDF, and at median $\frac{1}{2}$ it is exactly one half (to 10^{-8});
- for normal voters the pairwise shares match their closed form (Equation 39) (to 10^{-12}), and every Condorcet method draws the Voronoi diagram;
- the shares match 10^6 sampled voters within five standard errors (monte_carlo_check does the same for any pixels, with $2 \cdot 10^6$ voters, whose standard error is at most $1/(2\sqrt{M}) \approx 3.5 \cdot 10^{-4}$);
- interpolated shares match the exact ones at every pixel (to $2 \cdot 10^{-5}$ for Beta voters with 33 nodes, $2 \cdot 10^{-6}$ for normal voters with 49 nodes).

3. Voting methods (pixels/methods.py)

Every method receives the rankings (an $R \times C$ array, best first, one row per cell) and the shares $P(r)$ of every pixel (an $n \times n \times R$ array), and returns the winner of every pixel. Because an electorate is a distribution, a method works with shares instead of vote counts; every step is a matrix product over the R rankings, done for all pixels at once.

3.1. First past the post (fptp)

The first-choice share of candidate i is

$$s_i = \sum_{r:r_1=i} P(r), \quad (34)$$

and the candidate with the largest share wins.

3.2. Instant runoff (irv)

With a set S of eliminated candidates, every ballot counts for its highest ranked candidate outside S ,

$$s_i^S = \sum_{r: \text{first of } r \text{ outside } S \text{ is } i} P(r). \quad (35)$$

The candidate with the smallest share is added to S , and after $C - 1$ rounds the remaining candidate wins. The rankings are complete, so no ballot is ever exhausted and a candidate with a majority is never eliminated: running to the end gives the same winner as stopping at a majority. The rounds run in a compiled loop over the pixels (Section 7.4): every ranking points to its highest ranked candidate outside S , and eliminating a candidate moves only the rankings that point to it. All $C - 1$ rounds together take at most RC steps per pixel.

3.3. Borda count (borda)

With $\text{pos}_r(i) \in \{0, \dots, C - 1\}$ the position of i in ranking r ,

$$\text{score}_i = \sum_r P(r)(C - 1 - \text{pos}_r(i)), \quad (36)$$

and the highest score wins.

3.4. Pairwise majorities

Condorcet methods only use the pairwise shares

$$\pi_{ij} = \sum_{r: i \text{ above } j \text{ in } r} P(r), \quad \pi_{ij} + \pi_{ji} = 1, \quad (37)$$

the share of voters preferring c_i to c_j ; c_i beats c_j when $\pi_{ij} > \frac{1}{2}$.

Condorcet winner (condorcet_cycle). The candidate who beats every other candidate. If there is none the pixel is marked with CYCLE (black in the figures). As a function of the pixel, $\pi_{ij} = \frac{1}{2}$ holds only on a curve, so apart from pixel centres that happen to lie exactly on such a curve (Section 3.5) the majority relation is a tournament. A tournament in which nobody beats everyone always contains a cycle (a tournament without cycles is a strict order and has a top), so a black pixel is a Condorcet cycle unless it is such a tie.

Schulze (schulze). The defeat strength of c_i over c_j is π_{ij} if $\pi_{ij} > \pi_{ji}$ and 0 otherwise. The strength of the strongest path is found with the Floyd-Warshall recursion $p_{ij} \leftarrow \max(p_{ij}, \min(p_{ik}, p_{kj}))$ for $k = 1, \dots, C$, and the winner is a candidate with $p_{ij} \geq p_{ji}$ for all j . Schulze elects the Condorcet winner whenever there is one, so its diagram differs from the Condorcet winner diagram only on the black pixels, where it resolves the cycle.

3.5. Ties

Exact ties are broken by the order of the candidates: argmax and argmin take the first candidate. With exact shares a tie only happens on sets of zero area, e.g. for pixel centres exactly on a bisector under normal voters, where $\pi_{ij} = \frac{1}{2}$ and neither candidate beats the other (the dotted black lines in Figure 3).

4. Beta versus normal voters

With the same candidates the two voter models give visibly different diagrams, and the difference is largest where one would expect it least: for Condorcet methods. On votingmethods.net/ye (normal voters) the River method splits the square into one clean region per candidate, while Schulze on the Beta model has distorted regions and even pixels without a Condorcet winner (Figure 3). This chapter explains why.

All numbers below use the candidates A–E (Figure 2), the default spread rule `rms`, 300×300 pixels, 49×49 nodes and $D = 0.3$. This is more than the default 0.2 so that the effects are easy to see; at 0.2 they are the same in kind but smaller (Section 4.2.5). The numbers and figures are produced by `docs/figures.py`. How the other spread rules change them is the subject of Chapter 5.

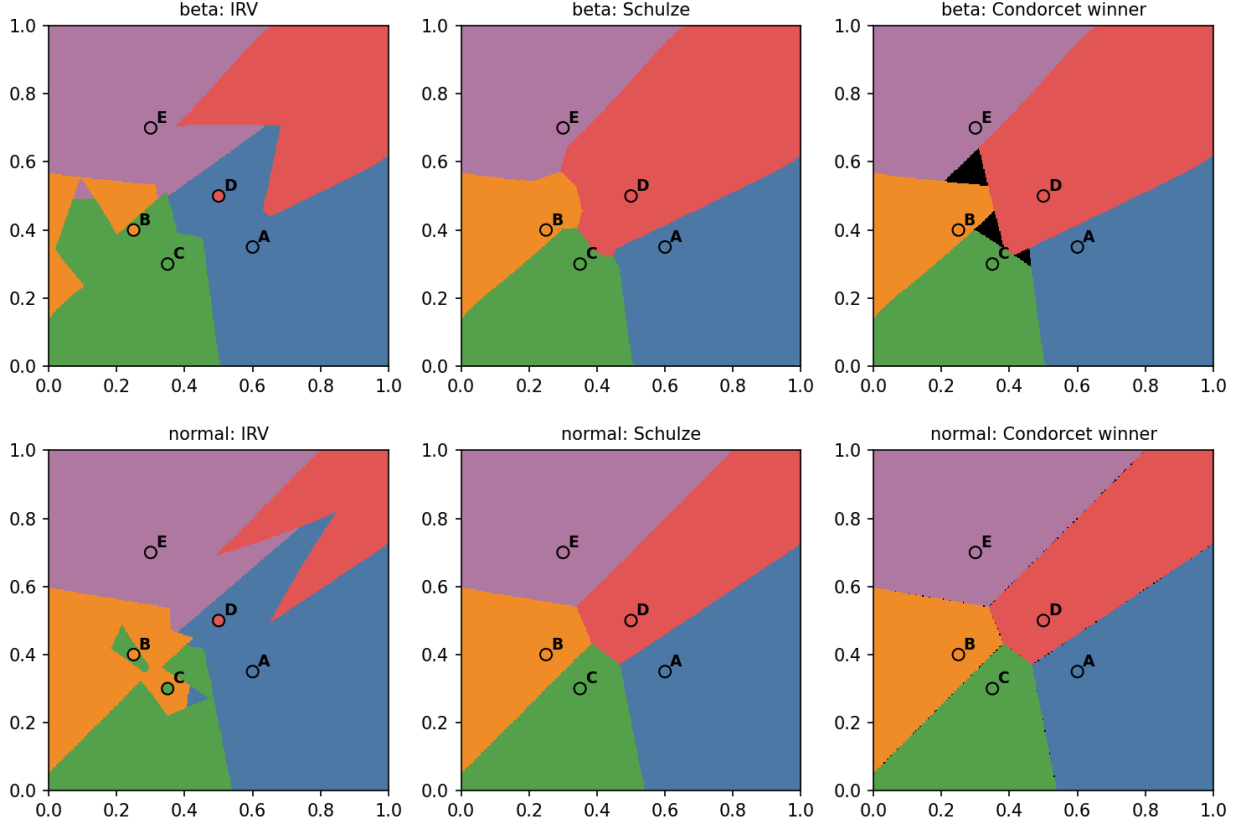


Figure 3: Beta voters (`rms`, top) and normal voters (bottom), $D = 0.3$. Normal Schulze and the normal Condorcet winner are exactly the Voronoi diagram of the candidates. Black: no Condorcet winner. The dotted black lines in the bottom right plot are pixels whose centre lies exactly on a bisector (a tie, not a cycle).

4.1. The two models

	Beta (<code>ranking_cells.py</code>)	normal (<code>normal.py</code>)
voters of pixel m	$X \sim \text{Beta}(a_x, b_x)$, $Y \sim \text{Beta}(a_y, b_y)$ independent	$(X, Y) \sim \mathcal{N}(m, \sigma^2 I)$
support	unit square	whole plane
m is	the median of each coordinate	mean and median, centre of symmetry
spread	$E X - \frac{1}{2} = D$ at the centre pixel, RMS from the median fixed (<code>rms</code>)	$E X - m_x = D$ everywhere, Equation 3
symmetric about m	only for $m = (\frac{1}{2}, \frac{1}{2})$	always

`votingmethods.net` uses a standard deviation of 0.4 by default ($D \approx 0.319$ here) and samples only 140 voters per pixel, so its diagrams also carry Monte Carlo noise (ragged IRV borders, small islands). The shares here are exact.

4.2. Pairwise majorities

By Equation 14 a voter at p prefers c_i to c_j exactly when $p \cdot \nu < \omega$ with $\nu = c_j - c_i$ and $\omega = (|c_j|^2 - |c_i|^2)/2$. The pairwise share (Equation 37) of the voters of pixel m who prefer c_i to c_j is therefore

$$\pi_{ij}(m) = P_m(X \cdot \nu < \omega), \quad (38)$$

and it depends only on the one-dimensional projection $X \cdot \nu$ of the voters onto the direction from c_i to c_j . Every Condorcet method only looks at these numbers.

4.2.1. Normal voters: the majority is decided by distance

The projection of an isotropic normal is normal, $X \cdot \nu \sim \mathcal{N}(m \cdot \nu, \sigma^2 |\nu|^2)$, so

$$\pi_{ij}(m) = \Phi\left(\frac{\delta_{ij}(m)}{\sigma}\right), \quad \delta_{ij}(m) = \frac{\omega - m \cdot \nu}{|\nu|}, \quad (39)$$

where $\delta_{ij}(m)$ is the signed distance of m from the bisector, positive on the side of c_i . A majority prefers c_i to c_j if and only if $\delta_{ij}(m) > 0$, that is, if and only if $|m - c_i| < |m - c_j|$. Consequently:

1. The majority relation of every pixel is the order of the candidates by distance from m . It is transitive: **there are no Condorcet cycles**.
2. The Condorcet winner always exists and is the candidate nearest to m , so **every** Condorcet method (Schulze, River, Ranked Pairs, Minimax, ...) draws exactly the Voronoi diagram of the candidates.
3. This holds **for every** σ . The only property used is that every line through m has half of the voters on each side, which is true for every distribution that is centrally symmetric about m .

This is why the River diagram of votingmethods.net looks like a Voronoi diagram; its borders agree with the bisectors (e.g. the D|A bisector meets $x = 1$ at $y = 0.725$ and the D|E bisector meets $y = 1$ at $x = 0.8$). The spread σ still matters for methods that use more than pairwise majorities (FPTP, IRV, Borda).

4.2.2. Beta voters: the median is a median only along the axes

For the Beta model m is the median of X and of Y separately. If ν is parallel to an axis, $X \cdot \nu$ is a multiple of one coordinate, its median is $m \cdot \nu$, and the line through m perpendicular to ν splits the voters exactly in half. For an oblique ν this fails: the median of $\nu_x X + \nu_y Y$ is in general **not** $\nu_x m_x + \nu_y m_y$, because medians are not additive for skewed variables. For every direction ν there is a different “effective centre” of the pixel, shifted from m towards the mean of the voters. Consequently:

1. The Condorcet winner is no longer the candidate nearest to m , so Condorcet regions are distorted.
2. Different pairs of candidates are compared from different effective centres, so the majority relation need not be transitive: **Condorcet cycles appear** (black in Figure 3). If some point were a median in every direction (a “total median”, Davis, DeGroot and Hinich, 1972), the argument of the normal case would apply with that point in place of m ; a product of skewed Beta marginals generally has none.

4.2.3. The shape of Beta voters

At $D = 0.3 > \frac{1}{4}$ the Beta is U-shaped even at the centre (Section 1.4). Towards a wall:

median	a	b	mean	$P(X < 0.1)$	$P(X > 0.9)$
0.50	0.602	0.602	0.500	0.175	0.175
0.70	0.743	0.454	0.621	0.095	0.298
0.90	0.796	0.265	0.750	0.051	0.500
0.98	0.648	0.151	0.812	0.049	0.640

As the median approaches the wall, half of the voters crowd into the strip between the median and the wall, while the other half forms a long tail towards the centre of the square. The mean lags behind the median by up to 0.17: the voters of every pixel off the centre are skewed towards the middle of the square, and so are their effective centres.

4.2.4. Example: the pull towards the centre

The pixels below lie on A's side of the bisector of $A = (0.6, 0.35)$ and the centre candidate $D = (0.5, 0.5)$; δ is their distance from it. Share of voters preferring A (Beta: $4 \cdot 10^6$ sampled voters):

pixel m	δ	normal	Beta	Beta mean
(0.95, 0.55)	0.118	0.623	0.525	(0.786, 0.530)
(0.90, 0.60)	0.049	0.551	0.473	(0.750, 0.560)
(0.80, 0.45)	0.118	0.623	0.541	(0.684, 0.470)

In all three pixels A gets far less support than under normal voters, and in the second the median voter is closer to A, yet D wins the pairwise contest: the long tails of the Beta point towards the middle of the square, and so does the effective centre.

4.2.5. The whole diagram

Leaving out the 357 pixels whose centre lies exactly on a bisector (where both candidates are correct winners):

	Schulze $\neq$ Voronoi	no Condorcet winner	area of D (Voronoi: 0.208)
Beta, $D = 0.3$	12 705 pixels (14.2 %)	1 101 pixels (1.23 %)	0.312
Beta, $D = 0.2$	4 980 pixels (5.6 %)	48 pixels (0.05 %)	0.247
normal, any D	0	0	0.208

The centre candidate D gains most: the Beta electorates of the whole upper right part of the square are pulled towards the centre, where D is.

4.3. Curved borders

Under normal voters the borders of Condorcet methods are bisectors, i.e. straight lines. The borders of FPTP and IRV are curved in both models, and the Beta model adds bends near the walls of the square. The key is to look at a first-choice share as a function of the pixel: it is the Voronoi cell of the candidate, **blurred**.

4.3.1. First-choice shares are blurred Voronoi cells

The FPTP winner changes where two first-choice shares (Equation 34) are equal,

$$s_i(m) = s_j(m), \quad s_i(m) = P_m(X \in V_i) = \int_{V_i} f_m(p) dp, \quad (40)$$

with V_i the Voronoi cell of c_i and f_m the voter density of pixel m . For normal voters $f_m(p) = \varphi_\sigma(p - m)$ is one Gaussian bump moved to m , and since it is symmetric,

$$s_i = \mathbb{1}_{V_i} * \varphi_\sigma, \quad (41)$$

the indicator of the cell (1 inside, 0 outside) convolved with a Gaussian of radius σ : the cell **blurred** like an out-of-focus photograph. The FPTP diagram is obtained by blurring every Voronoi cell and colouring each pixel by the brightest one; for $\sigma \rightarrow 0$ this is the Voronoi diagram itself. IRV does the same once per round, with the cells V_i^S of the candidates S still in the race. A border is therefore a **contour line** where two blurred cells are equally bright, and the question is when such a contour line is straight.

For Beta voters there is no single bump: f_m changes its shape with m (U-shaped, crowded against the walls, Figure 1 and Figure 7), so the blur itself varies across the square. This is the source of the bends near the walls below.

4.3.2. Straight versus round: blurring a corner

Half-plane: straight. With two candidates each cell is a half-plane, and by Equation 39 its blurred version is $\Phi(\delta/\sigma)$, a function of the distance δ from the bisector only. All its contour lines are parallel to the bisector, and the tie $s_i = s_j = \frac{1}{2}$ is the bisector itself.

Corner: round. Let a cell be the quadrant $Q = \{x < \frac{1}{2}, y < \frac{1}{2}\}$. The two coordinates of the normal are independent, so its blurred version is exactly

$$s_Q(m) = \Phi\left(\frac{\frac{1}{2} - m_x}{\sigma}\right) \Phi\left(\frac{\frac{1}{2} - m_y}{\sigma}\right). \quad (42)$$

Far from the corner one factor is 1 and the contour lines are straight lines parallel to the edges of Q . At the corner they bend round (Figure 4, middle). For example the contour $s_Q = \frac{1}{2}$ approaches the edges of Q far away, but on the diagonal it needs $\Phi(u)^2 = \frac{1}{2}$, $u = \Phi^{-1}(1/\sqrt{2}) = 0.545$: it passes $0.545\sqrt{2}\sigma \approx 0.77\sigma$ **inside** the corner. Blurring rounds the corner with a radius of the order of σ , exactly as blurring a photograph of a square gives a rounded square.

Vertices: the widest cell wins. A cell that is a wedge with angle α at a Voronoi vertex receives exactly $\alpha/(2\pi)$ of the voters of the pixel centred at that vertex (the Gaussian is rotationally symmetric). Where several cells meet, the one with the largest angle wins at the vertex, and the fronts of the others recede by a distance of the order of σ along round contour lines. Far from the vertices (distance $\gg \sigma$) only two cells are nearby and the border returns to their bisector.

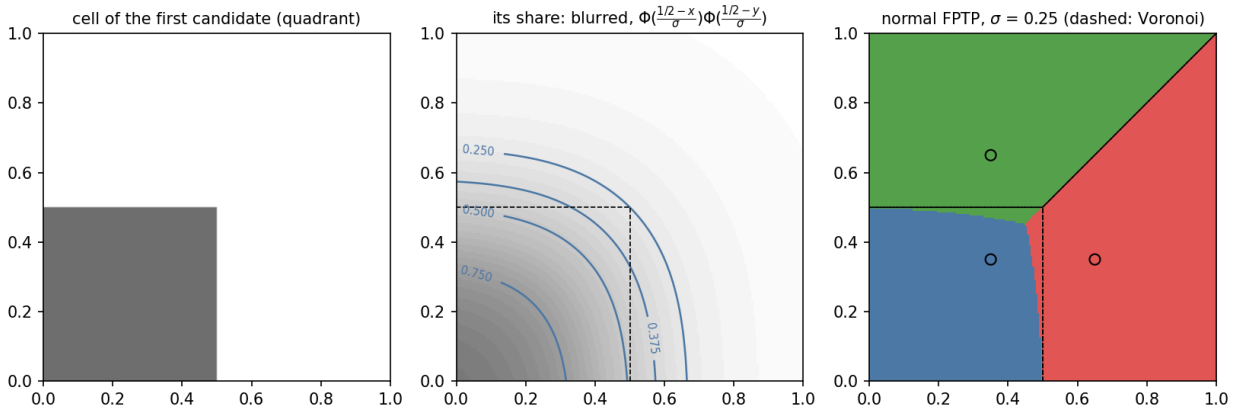


Figure 4: Three candidates whose cells meet at $(\frac{1}{2}, \frac{1}{2})$ with angles 90° , 135° , 135° (normal voters, $\sigma = 0.25$). Left: the quadrant cell of the lower-left candidate. Middle: its share (Equation 42) with contour lines. Right: FPTP. At the vertex the shares are $\frac{1}{4}$, $\frac{3}{8}$, $\frac{3}{8}$, so the quadrant candidate loses its corner, along a round front.

4.3.3. Seven candidates

Figure 5 shows FPTP for seven candidates (B removed, three added on the left). With a small blur ($D = 0.1$) the diagram is the Voronoi diagram with rounded corners at the vertices. With $D = 0.3$ the blur is wider than several cells:

- the centre candidate $(0.5, 0.5)$ has a thin band as its cell. Blurred, a thin band stays dim, so under normal voters it wins only a small patch in the far corner of its band, not even its own position;
- borders appear between candidates whose cells do not even touch, e.g. $(0.3, 0.7)$ and $(0.6, 0.35)$. Such a border has no bisector to follow at all: it is purely a contour line of two blurred cells, and it is curved.

The Beta diagram shows the same effects, and in addition the blur changes from pixel to pixel: the centre candidate, helped by the pull towards the centre, wins a large region in the upper right, and the borders bend where the shape of the voters changes fastest, near the walls.

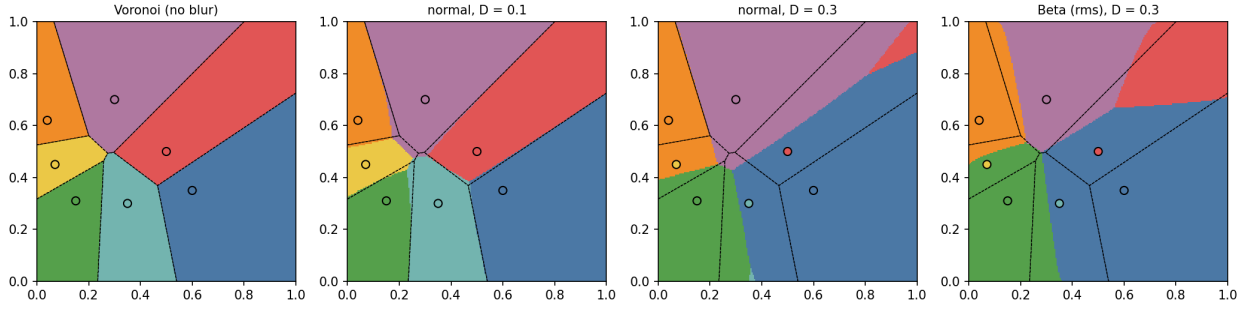


Figure 5: FPTP for seven candidates; dashed: Voronoi borders. From left: no blur (Voronoi), normal voters with $D = 0.1$ and $D = 0.3$, Beta voters (rms) with $D = 0.3$.

4.3.4. Flares at the walls (FPTP)

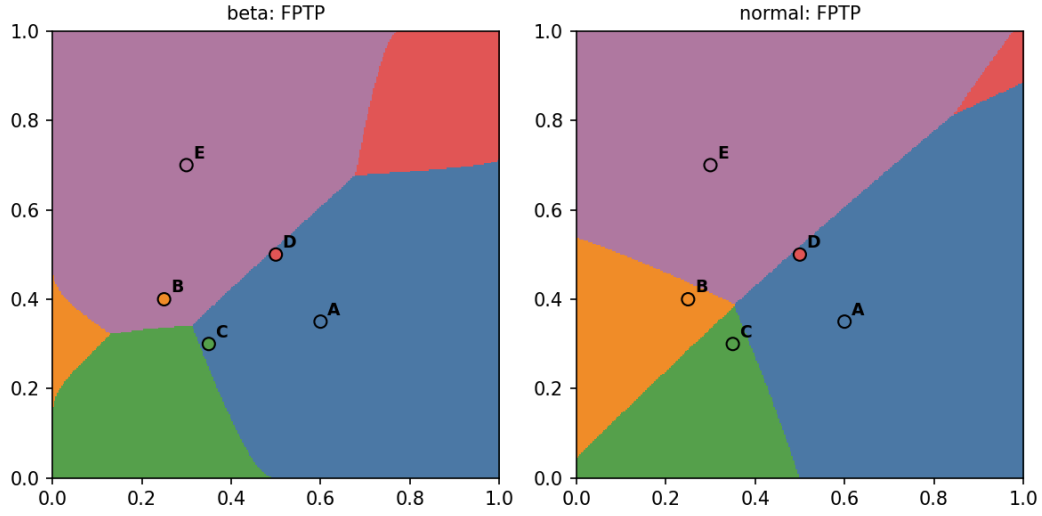


Figure 6: FPTP. With Beta voters the region of B flares out in the last few hundredths before the left wall; with normal voters it widens smoothly.

$B = (0.25, 0.4)$ is the leftmost candidate and its Voronoi cell widens towards the left wall: the bisector $B|C$ ($y = x + 0.05$) falls as x decreases, and at $x = 0$ the cell spans $0.05 < y < 0.596$. The more voters sit close to $x = 0$, the more of them fall into the wide part of B's cell. Heights y of the borders of B's region in the column of median x ("–": B wins nowhere in the column):

median x	$P(X < 0.02)$	Beta B E	Beta C B	normal B E	normal C B
0.200	0.212	–	–	0.455	0.238
0.080	0.357	0.355	0.272	0.505	0.125
0.030	0.467	0.398	0.218	0.525	0.075
0.005	0.581	0.438	0.172	0.535	0.048

With Beta voters B's region only begins near the wall and then widens fast: over the last 0.075 the B|E border rises by 0.083, against 0.030 with normal voters. Figure 7 shows why. As the median approaches the wall, the half of the Beta voters on the wall side is squeezed into an ever thinner strip next to the wall, right into the wide part of B's cell ($P(X < 0.02)$ grows from 0.21 to 0.58), and B's share grows from 0.22 to 0.33 between $x = 0.2$ and $x = 0.02$. The normal cloud keeps its shape and only moves; its left half continues beyond $x = 0$, where B's cell simply keeps widening, so nothing piles up at the wall.

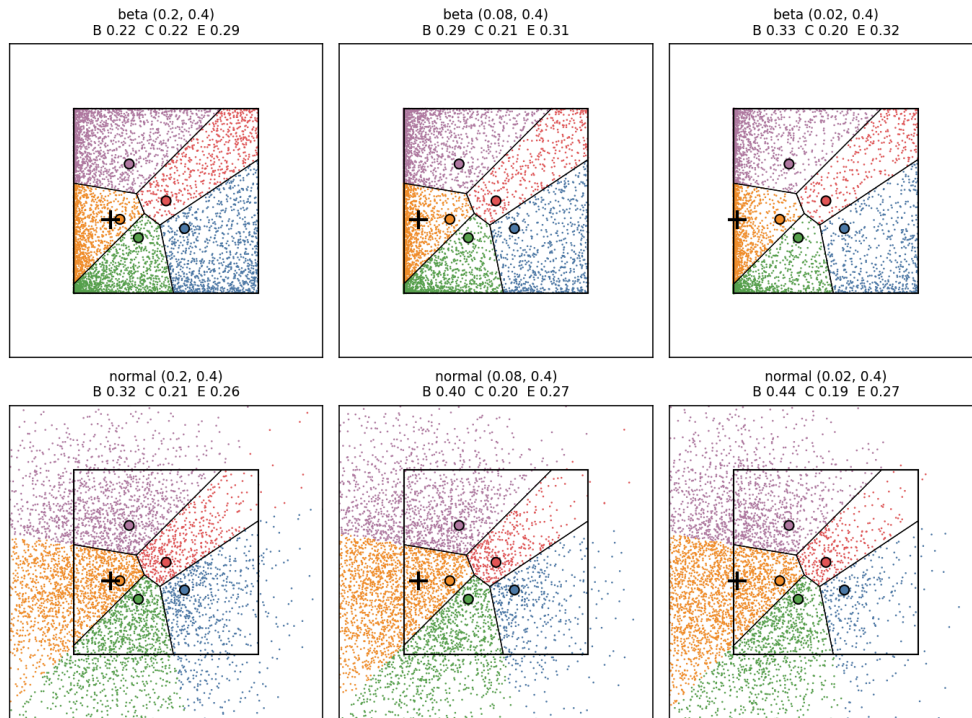


Figure 7: Voters of the pixels $(x, 0.4)$ approaching the left wall, coloured by their first choice (FPTP), with the Voronoi borders; titles: shares of B, C and E.

The flare is a property of any wide Beta, not of the spread rule: it is nearly the same under tapered and mean_abs (Chapter 5). It fades with less spread: at $D = 0.2$ the B|E border rises by only 0.030, like normal voters at $D = 0.3$.

4.4. Summary

- Normal voters are symmetric about the pixel centre, so pairwise majorities follow distance: no cycles, and every Condorcet method draws the Voronoi diagram, whatever σ .
- Beta voters have the pixel centre as the median of each coordinate only. Majorities between candidates on a diagonal are decided by the skew of the distribution, which pulls the effective centre towards the middle of the square: Condorcet regions are distorted (14 % of the pixels at $D = 0.3$, 6 % at $D = 0.2$) and cycles appear.
- A first-choice share is the candidate's Voronoi cell blurred by the voter distribution. Blurring keeps half-planes straight but rounds corners (radius of order σ), so FPTP and IRV borders are curved in both models.
- Beta voters change shape across the square and crowd against the walls, so the blur varies from pixel to pixel; near the walls this produces flares such as B's region in FPTP.

5. Choosing the spread rule

Section 1.5 defined three spread rules. They agree at the centre pixel and differ only in how the spread changes towards the walls. This chapter shows what that changes in the diagrams, explains which property of a rule keeps borders straight, and gives the evidence for the default rms. Unless stated otherwise the numbers use the candidates A–E, $D = 0.3$ and 300×300 pixels. `docs/figures.py` reproduces the figures and the numbers on the default candidates; the benchmark over random candidate layouts (Section 5.4) was run with separate scripts.

5.1. The three rules on the default candidates

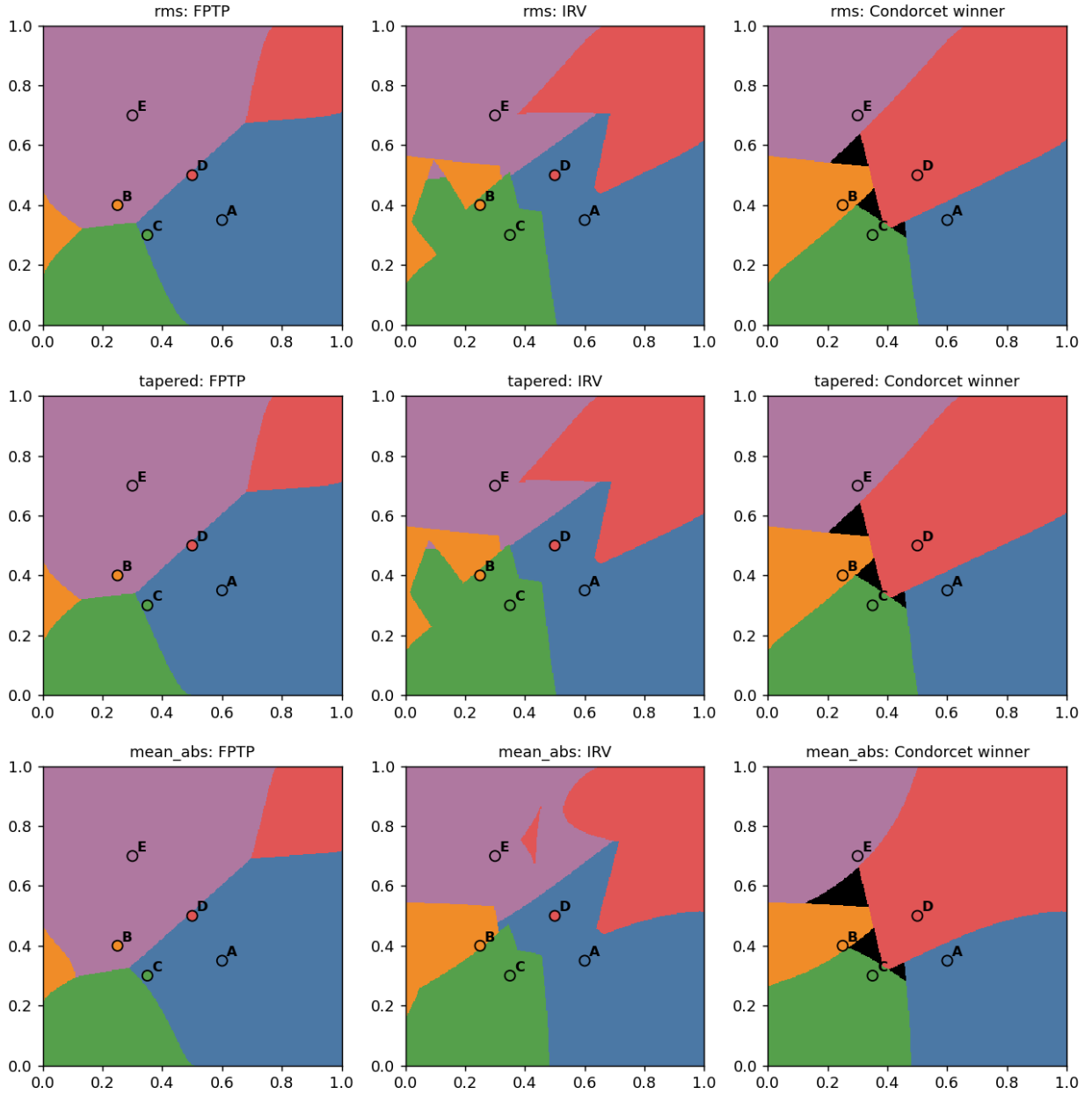


Figure 8: The three spread rules of Section 1.5 on the default candidates, $D = 0.3$. Only mean_abs has the round IRV edge. Black: no Condorcet winner.

voter model	cycle pixels	Schulze $\neq$ Voronoi	round IRV edge	flare of B
Beta, rms	1.23 %	14.2 %	no	0.083
Beta, tapered	1.32 %	14.9 %	no	0.086
Beta, mean_abs	2.06 %	20.2 %	yes	0.096
normal	0	0	no	0.030

(Flare of B: rise of the B|E border over the last 0.075 before the left wall, Section 4.3.4.) rms and tapered give nearly the same diagrams. mean_abs has 1.7 times as many cycle pixels, more distorted Condorcet regions and a round IRV edge. The FPTP flare is almost the same under all three rules: it comes from the crowding of voters at the wall, which does not depend on the rule.

5.2. The round IRV edge of mean_abs

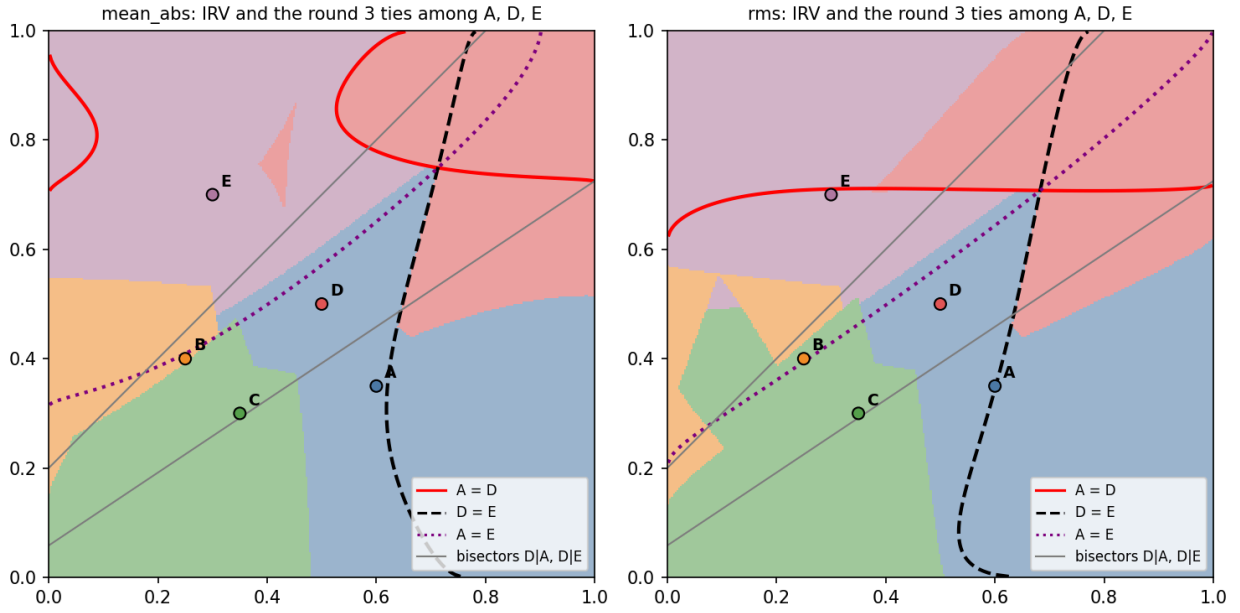


Figure 9: Beta IRV (faded) with the three round 3 tie curves among A, D, E, under mean_abs (left) and rms (right).

The red curve $s_A = s_D$ bounds D's region. Gray: the bisectors D|A and D|E.

In the upper right part B and C are eliminated first. In round 3 A, D and E remain, and D's cell among them is the diagonal band between the bisectors D|A and D|E that widens towards the corner (1, 1); A has the bottom right, E the top left. Whoever of A and D has fewer votes is eliminated:

- D out: the final is A against E, and E wins;
- A out: the final is D against E, and D wins near the band (the pull towards the centre again).

So the lower edge of D's region is the round 3 tie $s_A = s_D$ (red in Figure 9). Along the column $x = 0.55$ (round 3 shares; winner of the pixel):

mean_abs					rms				
y	winner	s_A	s_D	$P(Y < 0.2)$	winner	s_A	s_D	$P(Y < 0.2)$	
0.678	E	0.327	0.284	0.195	E	0.316	0.292	0.173	
0.738	E	0.303	0.286	0.185	D	0.279	0.300	0.146	
0.798	E	0.285	0.284	0.182	D	0.243	0.307	0.123	
0.858	D	0.274	0.279	0.187	D	0.209	0.311	0.102	
0.918	D	0.270	0.270	0.199	E	0.176	0.312	0.087	
0.978	E	0.274	0.254	0.226	E	0.144	0.307	0.079	

Under rms, going up the column, voters leave A's cell for D's band and E's part of the square, s_A falls steadily and the tie $s_A = s_D$ is crossed once, along a nearly straight curve. (At the top D still survives round 3 but loses the final to E.) Under mean_abs s_A stops falling and rises again: as the median of Y approaches 1, the fixed mean absolute deviation pushes the lower half of the voters down (Equation 9), $P(Y < 0.2)$ grows from 0.182 to 0.226, and those voters are in A's cell. D is ahead of A only in a middle band, so the tie curve closes around it into the round edge.

5.3. What keeps a border straight

Every rule is a curve $\kappa(m)$ (Section 1.5). The border between c_i and c_j is where the median of the projected voters $\nu \cdot X$ equals the bisector value. That median lies at some distance from $\nu \cdot m$, the pull of the pixel towards the centre, and the border is straight when the pull changes linearly along it.

Borders close to an axis (exact). For a small tilt ε , $F_{X+\varepsilon Y}(t) = E[F_X(t - \varepsilon Y)] = F_X(t) - \varepsilon f_X(t)E[Y] + O(\varepsilon^2)$, so

$$\text{median}(X + \varepsilon Y) = \text{median}(X) + \varepsilon E[Y] + O(\varepsilon^2), \quad (43)$$

and the pull is $\varepsilon(E[Y] - \text{median}(Y))$. Such borders are straight exactly when median minus mean grows in proportion to the distance of the median from the centre, i.e. when $(m - EX)/(m - \frac{1}{2})$ is constant. No small-spread assumption is needed (checked numerically). This ratio for several rules:

rule	$m = 0.6$	0.7	0.8	0.9	0.98
fixed $a + b$	0.40	0.39	0.38	0.35	0.27
rms	0.40	0.40	0.39	0.37	0.35
tapered, $\tau = 0.2$	0.40	0.40	0.40	0.39	0.37
tapered, $\tau = 0.25$	0.40	0.41	0.41	0.41	0.40
mean_abs	0.41	0.44	0.48	0.54	0.59

There are two ways to fail. With a fixed $a + b$ the far tail thins out near a wall, the mean catches up with the median and the pull stalls. With a fixed $E|X - m|$ the far half is pushed away (Equation 9) and the pull accelerates. rms and tapered lie in between, close to constant.

Diagonal borders. At 45° both coordinates are skewed at once. Exact border shapes bend one way for fixed $a + b$ and the other way for fixed $E|X - m|$, confirming the two failure modes, but the balance point moves with position: near the centre a smaller τ is straighter, towards the corners $\tau \approx 0.2$ – 0.25 . Near-axis borders prefer 0.25, diagonal ones near the centre less, and the benchmark below puts the compromise at 0.2. Above 0.25 both kinds bend the wrong way, which is why the optimum is sharp. The textbook skewness approximation of the median is off by about a factor of two at $D = 0.3$, so the exponent has to be found empirically.

Why cycles survive. Straightness depends on how the pull changes with the position of the pixel, cycles on how it changes with the direction ν . Projections of two skewed coordinates in different directions do not share one effective centre, so the three borders of three candidates can each be straight and still miss each other. No spread rule removes cycles without removing the spread.

5.4. Benchmark

The **bend** of a rule is the largest distance of each pairwise majority border $\pi_{ij} = \frac{1}{2}$ from its chord, averaged over the pairs (units of the square; 0 for normal voters). At the same $D = 0.3$, on the default candidates:

rule	$\kappa(0.7)$	$\kappa(0.9)$	$\kappa(0.98)$	bend	cycle pixels
mean_abs (L_1)	1.012	0.554	0.307	0.046	2.06 %
$L_{1.5}$ from median	1.096	0.807	0.557	0.015	1.55 %
rms (L_2)	1.197	1.062	0.799	0.009	1.23 %
L_3 from median	1.430	1.546	1.250	0.010	0.82 %
L_4 from median	1.684	1.990	1.667	0.011	0.59 %
fixed $a + b$	1.204	1.204	1.204	0.024	1.21 %
SD about the mean	1.086	0.779	0.557	0.015	1.59 %
far-side mean distance	1.826	1.624	1.045	0.012	0.43 %
far-side RMS	2.112	2.078	1.514	0.010	0.31 %
far-side median distance	1.444	1.096	0.622	0.026	0.75 %
SD of logit(X)	1.320	2.262	8.106	0.038	0.98 %

“Far-side” measures use only the half of the voters towards the centre of the square. Too little decrease of κ (fixed $a + b$) and too much (mean_abs) both bend borders. But a comparison at the same D mixes the rule with the amount of spread: cycles and bend both grow with the spread, and a rule that makes pixels near the walls narrower (larger κ) gets fewer cycles and less bend together. The fair comparison is the bend at the **same share of cycle pixels**.

The benchmark therefore swept $D \in \{0.2, 0.25, 0.3, 0.35, 0.4\}$ for every rule and compared the bend at equal shares of cycle pixels, interpolating in D . The score is the bend of the *visible* Condorcet borders (where both candidates beat all others), on layouts of 3–6 candidates drawn uniformly from $[0.1, 0.9]^2$. A first screen on 24 layouts ruled out most rules (bend relative to rms): fixed $a + b$ 2.83, mean_abs 2.41, RMS of arcsin $\sqrt{X}$ 2.00, $\tau = 0.1$ 1.92, far-side RMS 1.84, L_3 1.35, $L_{2.5}$ 1.21, $L_{2.25}$ 1.10. The remaining rules were checked on 30 new layouts, with 95 % bootstrap intervals over layouts, at three cycle levels (those of rms at $D = 0.25, 0.3, 0.35$):

rule	$D = 0.25$ level	$D = 0.3$ level	$D = 0.35$ level
tapered, $\tau = 0.2$	0.66 [0.62, 0.70]	0.63 [0.59, 0.70]	0.56 [0.49, 0.65]
tapered, $\tau = 0.25$	0.76 [0.67, 0.87]	0.90 [0.75, 1.07]	1.03 [0.81, 1.28]
tapered, $\tau = 0.3$	1.52	1.97	2.14
tapered, $\tau = 0.4$	3.55	4.49	4.30
tapered, $\tau = 0.5$	5.94	7.25	6.80
$L_{1.75}$	1.01 [0.96, 1.05]	0.92 [0.88, 0.98]	0.77 [0.73, 0.80]
$L_{1.5}$	1.14 [1.05, 1.24]	0.94 [0.83, 1.08]	0.67 [0.60, 0.77]

The whole pairwise borders (visible or not) give the same picture: $\tau = 0.2$ at 0.79, 0.79 and 0.52 of rms. In absolute terms the gain is small: at $D = 0.3$ visible borders bend by 1.5 pixels on average with rms and 1.0 with tapered (90th percentile 3.1 and 2.0 pixels; the single worst border 4.9 and 7.7 pixels).

5.5. Other ways to fix the spread

- **Median absolute deviation.** For every Beta, median $|X - m| < \min(m, 1 - m)$: the near half lies entirely within that distance. A fixed median absolute deviation is therefore impossible within D of every wall.
- **Normal voters clamped to the square** (moved onto the nearest edge when they would leave it) have neither the round IRV edge nor a strong flare (0.023), few cycles (0.10 %) and Schulze $\neq$ Voronoi on 7.9 % of the pixels. But they put point masses on the edges, are not a Beta, and their deviation shrinks near the walls.

5.6. Conclusion

- **rms** is the default. It fixes a quantity with a clear meaning, pushes the far half of the voters out only by $\sqrt{2}$ near a wall, has no round IRV edge, and its borders bend only slightly more than those of the best rule found (1.5 against 1.0 pixels on average).
- **tapered** keeps Condorcet borders a little straighter at the same number of cycles and otherwise behaves like rms; its exponent is empirical.
- **mean_abs** is kept to reproduce results made with it (caches and plots are kept apart per rule, $\text{cache}/\text{beta}/\text{<spread>}/$, $\text{plots}/\text{beta}/\text{<spread>}/$). By Equation 9 it pushes the far half of the voters out twice as far near a wall, which bends borders the most, adds cycles and makes the round IRV edge.

6. Shapes of win regions

A rule of thumb about Yee diagrams links the shape of the regions with monotonicity: methods that are not monotone, such as IRV, draw regions that are not convex and sometimes fall into several pieces. This chapter makes the link precise. The shape of a region is decided by the method and the voter model together, and the rule of thumb holds only in special cases. Monotone methods can have non-convex regions and regions in several pieces, and non-monotone methods can have convex regions.

6.1. Regions, shapes and monotonicity

The **win region** R_i of candidate c_i is the set of pixel centres $m \in (0, 1)^2$ where c_i wins. Apart from exact ties, which only happen on curves (Section 3.5), every method decides its winner by strict inequalities between continuous functions of m : first-choice shares, pairwise shares, IRV totals or Schulze path strengths. So R_i is an open set. Three shapes, from the strongest to the weakest:

- R_i is **convex** if the segment between any two of its points lies in R_i .
- R_i is **star-shaped** around a point q if the segment from q to any point of R_i lies in R_i . Star-shaped around the candidate's own position is the natural property between the other two.
- R_i is **connected** if it is not the union of two disjoint non-empty open sets. For an open set in the plane this is the same as **path-connected**: any two of its points are joined by a continuous path inside it.

A convex region is star-shaped around each of its points, and a star-shaped region is connected. On the pixel grid a region is **in pieces** when its pixels fall into groups that do not touch, not even at a corner.

Curved borders. Two neighbouring regions can both be convex only if the border between them is straight: two disjoint open convex sets are separated by a line (the separation theorem), so their common border lies on that line. Hence **a curved border always has a non-convex region on at least one side**, and every method that draws curved borders has non-convex regions.

Monotonicity. A method is **monotone** if a winner stays the winner when some voters raise them on their ballots and the order of the other candidates stays the same. FPTP, Borda and Schulze are monotone, and a Condorcet winner stays the Condorcet winner when raised. IRV is not monotone: raising the winner can change who is eliminated first, and so give the winner a stronger opponent in the final.

6.2. Why monotonicity does not fix the shape

Monotonicity compares two electorates that differ in one way only: some voters raise one candidate. Moving the pixel is a different change. Every voter moves, and the rankings change among all candidates, including pairs that do not involve the winner. Even moving the pixel towards a candidate is not the same as raising that candidate, because the voters beyond the candidate move away from it. So monotonicity says nothing about the winners of neighbouring pixels.

Another property that might seem to fit does not help either. A **consistent** method (FPTP and Borda are consistent) that elects c_i in two electorates also elects c_i when the two electorates are pooled. But the voters of the midpoint between two pixels are not the pooled voters of the two pixels: for normal voters they are one Gaussian bump at the midpoint, not two bumps.

6.3. Where the regions are convex

6.3.1. Condorcet methods and symmetric voters

For normal voters, and more generally for voters that are centrally symmetric about the pixel centre, every Condorcet method draws the Voronoi diagram (Section 4.2.1), whose regions are convex polygons. This includes Condorcet methods that are not monotone, such as Baldwin's and Nanson's methods: they elect the Condorcet winner whenever there is one, and for these voters there always is one. So a non-monotone method can have convex regions. Their convexity comes from the Condorcet criterion together with the symmetry of the voters, not from monotonicity.

6.3.2. Candidates on a line: FPTP versus IRV

Let the candidates lie on a line with unit direction u . Their Voronoi cells are strips perpendicular to the line, $V_i = \{p : p \cdot u \in I_i\}$, with intervals $I_1 < I_2 < \dots$ in the order of the candidates. For normal voters the projection $X \cdot u$ is $\mathcal{N}(t, \sigma^2)$ with $t = m \cdot u$, so every share depends only on t , and every region is a union of strips.

FPTP regions are single strips. For $t < t'$ the ratio of the two projected densities,

$$L(x) = \frac{\varphi((x - t')/\sigma)}{\varphi((x - t)/\sigma)} = \exp\left(\frac{(t' - t)x - (t'^2 - t^2)/2}{\sigma^2}\right), \quad (44)$$

increases in x . Moving the pixel from t to t' multiplies the share of cell k by the average of L over I_k , and for $i < j$ the interval I_j lies to the right of I_i , so the share of c_j grows by a larger factor than that of c_i : the ratio $s_j(t)/s_i(t)$ never decreases. Hence c_i beats c_j on a half-line of t , to the left of some point for $j > i$ and to the right of some point for $j < i$. The FPTP region of c_i is the intersection of these half-lines, one interval of t : a strip, which is convex. The argument only uses that L increases, which holds whenever the voters of a pixel are one fixed log-concave density moved to the pixel. Beta voters are not of this kind, and near the walls they are not even stochastically ordered (Section 6.6), so the argument does not carry over to them.

IRV splits the middle candidate. Take $L = (0.2, 0.5)$, $M = (0.5, 0.5)$ and $R = (0.8, 0.5)$ with normal voters, $D = 0.3$ ($\sigma = 0.376$). At the centre, M is the nearest candidate and the Condorcet winner, but it has the fewest first choices, $2\Phi(0.15/\sigma) - 1 = 0.310$ against 0.345 each for L and R . So M is eliminated first: the *centre squeeze*. At $x = 0.4$ the shares are L 0.445, M 0.300 and R 0.254, so R is eliminated first and M beats L in the final. M therefore wins IRV on two strips, $0.35 < x < 0.46$ and $0.54 < x < 0.65$, but not between them, and the regions of L and R are in two pieces as well (Figure 10). FPTP gives L and R one strip each and M nothing.

Here the rule of thumb is exactly right: the monotone method keeps every region in one piece and the non-monotone one does not. Both properties of IRV have the same cause: who meets whom in the final depends on who is eliminated first.

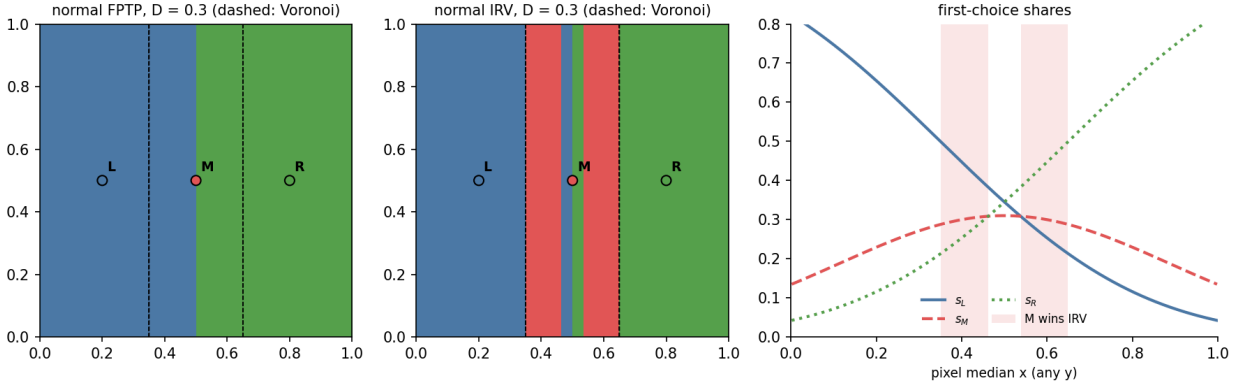


Figure 10: Three candidates on a line, normal voters, $D = 0.3$. FPTP gives every candidate at most one strip. In IRV the middle candidate M is squeezed out near the centre, where it has the fewest first choices (right), so M wins two separate strips, and so do L and R .

6.4. Monotone methods with non-convex regions

How often. The random layouts below count a region as non-convex when its convex hull contains pixels that are more than 3 pixels away from the region; pixelation alone cannot cause that. Table 1 counts, on 100 random layouts of 3–6 candidates in $[0.05, 0.95]^2$ (150×150 pixels, docs/figures.py search), the layouts with at least one such region, and the layouts with a region in pieces.

method	monotone	normal voters		Beta voters (rms)	
		non-convex	in pieces	non-convex	in pieces
FPTP	yes	42 / 21	0 / 0	34 / 47	0 / 0
Borda	yes	35 / 24	0 / 0	53 / 52	0 / 0
Schulze	yes	0 / 0	0 / 0	8 / 49	0 / 0
IRV	no	90 / 95	25 / 45	94 / 98	26 / 45

Table 1: Layouts out of 100 with at least one non-convex region, or with a region in pieces (more than one piece of at least 20 pixels), at $D = 0.2 / D = 0.3$.

Normal voters. The FPTP and Borda borders are contour lines of blurred cells and are curved (Section 4.3), so by the argument above some of their regions are not convex. The dent is often only a few pixels deep. Schulze draws the Voronoi diagram and is always convex.

Beta voters and Schulze. Under Beta voters Schulze regions are not convex either. Figure 11 shows an example with $c_1 = (0.457, 0.407)$, $c_2 = (0.254, 0.289)$, $c_3 = (0.334, 0.641)$ and $c_4 = (0.235, 0.464)$, rms and $D = 0.3$. c_1 wins at both ends of the segment, c_4 in most of it. The notch is not a bend of a pairwise border: along the whole segment c_4 beats c_1 head to head, with 0.508 to 0.516 of the voters, and in the middle c_4 is the Condorcet winner. The two ends lie in pockets of Condorcet cycles; in each the fourth candidate loses to the other three:

- at the upper end $c_1 > c_3 > c_4 > c_1$, with 0.512, 0.528 and 0.508 of the voters;
- at the lower end $c_1 > c_2 > c_4 > c_1$, with 0.530, 0.537 and 0.516.

In a cycle of three candidates Schulze elects the candidate beaten by the weakest of the three defeats. Next to the tie line of c_1 and c_4 the weakest defeat is that of c_1 by c_4 , barely above one half. So Schulze gives c_1 the part of each pocket along that line, on c_4 's side of it. In general Schulze splits a cycle pocket among its three candidates, each winning the part next to the tie line of its own defeat. Every Schulze region therefore reaches across its pairwise borders into the neighbouring pockets, and between two such pockets the region has a notch. Normal voters have no pockets: their three pairwise borders meet in one point, the Voronoi vertex.

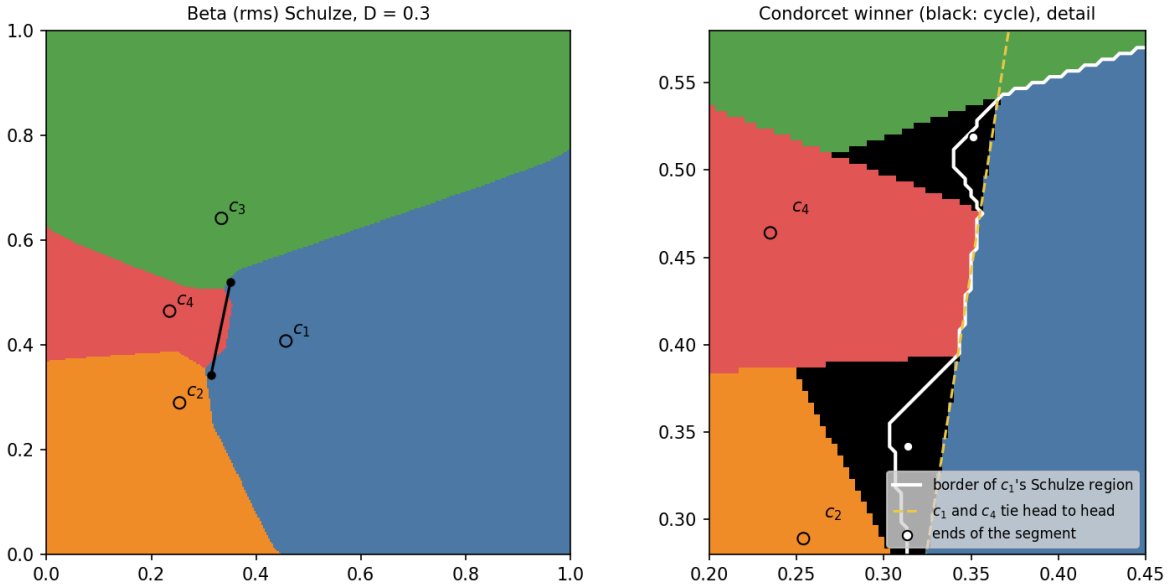


Figure 11: Left: Schulze with Beta voters (rms, $D = 0.3$). c_1 (blue) wins at both ends of the segment, c_4 (red) in between. Right: detail with the Condorcet winner (black: cycles). c_1 's Schulze region, to the right of the white line, reaches across the tie line of c_1 and c_4 (yellow) into two cycle pockets.

6.5. Monotone methods with regions in pieces

FPTP can split a region, already under normal voters. Take seven candidates: $A = (0.5, 0.5)$; $N = (0.5, 0.6)$ and $S = (0.5, 0.4)$ just above and below it; and $NW = (0.3, 0.95)$, $NE = (0.7, 0.95)$, $SW = (0.3, 0.05)$, $SE = (0.7, 0.05)$ far above and below. A 's Voronoi cell is the thin strip $0.45 < y < 0.55$.

With normal voters and $D = 0.12$ ($\sigma = 0.150$), N and S are strongest in the middle column: at A 's own position A has 0.260 of the first choices and N and S have 0.320 each, and along the whole column $x = \frac{1}{2}$ A trails the best rival by at least 0.060. Further out, the voters above the strip are split between N and NW or NE , and those below between S and SW or SE . Every rival is weaker there, and A wins. So A 's region is in two pieces, $0.13 < x < 0.28$ and $0.72 < x < 0.87$ (both at $0.46 < y < 0.54$), separated by the column where A never wins (Figure 12). Beta voters (rms, same D) give the same picture, with pieces at $0.08 < x < 0.25$ and $0.75 < x < 0.92$; along the column A trails by at least 0.081.

A does not win at its own position, so its region is not even star-shaped around A . The regions of NW , NE , SW and SE are not convex either: the domes of N and S bite into them. Layouts like this one are rare. In the random layouts of Table 1 no region of a monotone method is in pieces, while IRV has regions in pieces in a quarter to a half of them.

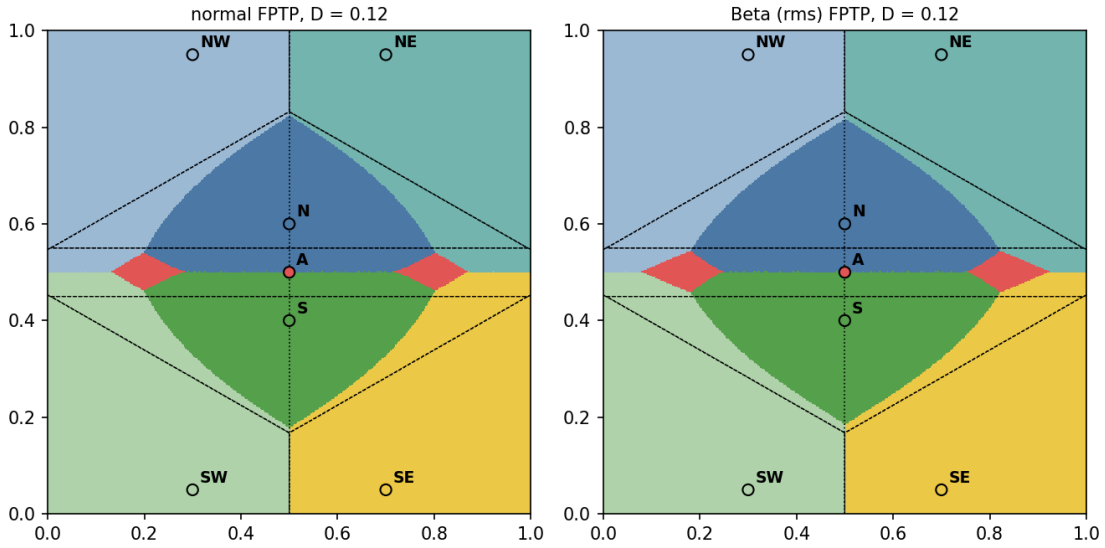


Figure 12: FPTP with normal and Beta voters, $D = 0.12$. Dashed: Voronoi borders; dotted: the column $x = \frac{1}{2}$, where A (red) never wins. A 's region is in two pieces.

6.6. Condorcet methods under Beta voters

For Condorcet methods under Beta voters, the random layouts gave non-convex regions but never regions in pieces. Whether Beta voters can split a Condorcet region is open. This section collects what is known.

Two candidates. Every method then elects the majority winner, and the region of c_i is $\{\pi_{ij} > \frac{1}{2}\}$, bounded by one bent border. Suppose that the voters of a pixel become stochastically larger in each coordinate as its median grows, i.e. that $P(X \leq t)$ never increases with the median, for any t . Then $\nu \cdot X$ becomes stochastically larger as m_x grows if $\nu_x > 0$ and smaller if $\nu_x < 0$, and likewise for m_y . So π_{ij} is monotone in each coordinate of the pixel, and with every pixel the region contains the whole rectangle between that pixel and one corner of the square. Any two points of the region are then joined through that corner: the region is connected, and its border is the graph of a monotone function. Normal voters have this property, because they only shift.

Beta voters do not quite have it. As the median moves towards a wall, the far half of the voters is pushed out (Section 1.5), and $P(X \leq t)$ grows for some t . The table gives, for medians up to $1 - 1/800$ (the last pixel of a 400×400 grid), the median from which $P(X \leq t)$ first grows, and the largest increase of $P(X \leq t)$ between two medians, over all t .

rule	$D = 0.2$		$D = 0.3$	
	grows from	largest rise	grows from	largest rise
rms	0.773	0.011	0.900	0.018
tapered	0.885	0.013	0.884	0.036
mean_abs	0.673	0.097	0.654	0.146

So the argument does not apply. Numerically, no two-candidate region was in pieces: 0 of 900 layouts (9 directions of the bisector times 100 positions) for each of rms and mean_abs at $D = 0.2, 0.3$ and 0.4 .

More candidates. The Condorcet region of c_i is the intersection of its pairwise regions. For it to fall into pieces, two of its borders would have to cross each other twice. They are bent by only a few pixels, so this would need two nearly parallel borders close together.

6.7. Summary

- Convex regions come from the voter model and the method together. Under normal voters every Condorcet method, monotone or not, draws convex regions, and FPTP does for candidates on a line.
- Monotone methods can have non-convex regions (FPTP and Borda under both models, Schulze under Beta voters, Figure 11) and regions in pieces (FPTP, Figure 12).
- So neither the shape nor the connectedness of a region proves anything about monotonicity. What holds is weaker: in the random layouts only IRV split regions, and for candidates on a line FPTP can never split a region, while IRV does.

7. Real-time diagrams in the web UI

The web UI redraws the diagram while a candidate is dragged. Chapter 2 computes the share of every ranking for every pixel, which is exact and serves every method, but takes too long for that:

step ($D = 0.2$, rms, 49×49 nodes)	5 candidates	8 candidates
ranking shares at the nodes (compute_ranking_probabilities)	333 ms	2878 ms
bisector arrangement (ranking_cells)	8 ms	99 ms
interpolation to 300×300 pixels	12 ms	88 ms
IRV / Schulze on the pixels	61 / 64 ms	703 / 358 ms

Almost all of the time goes to the shares at the $N \times N$ nodes, which do not depend on the number of pixels at all. Drawing the diagram as curves instead of pixels (Section 7.5) is the right output, but on its own it saves little. The speed comes from computing fewer integrals (Section 7.1), keeping those a drag does not change (Section 7.2), making each one cheaper (Section 7.3), and compiling the loops that remain (Section 7.4). This chapter describes `shares.py`, `beta_tables.py` and `regions.py`, which `web/app.py` uses for the UI. `docs/figures.py`, the plots and the tests of Section 2.6 still use the pipeline of Chapter 2, which is in `pixels/`.

7.1. What each method needs (shares.py)

The arrangement of all $C(C-1)/2$ bisectors has $O(C^4)$ cells and edges: 43 cells and 65 slanted edges for 5 candidates, 272 cells and 468 slanted edges for 8. Most methods need much less than the share of every cell.

Pairwise shares. Schulze and the Condorcet winner only use π_{ij} (Equation 37). Borda does too: $C-1 - \text{pos}_r(i)$ is the number of candidates below c_i in ranking r , so

$$\text{score}_i = \sum_r P(r)(C-1 - \text{pos}_r(i)) = \sum_{j \neq i} \pi_{ij}. \quad (45)$$

Each π_{ij} is the share of one half-plane, the part of the square on c_i 's side of the bisector: a convex polygon with one slanted edge. For 8 candidates that makes 28 slanted edges instead of 468. For normal voters no integral is needed at all: $\pi_{ij} = \Phi(\delta_{ij}(m)/\sigma)$ (Equation 39).

First-choice shares. FPTP only uses s_i (Equation 34), the share of the Voronoi cell of c_i : C convex polygons with about $3C$ edges in all. For normal voters the cells are taken in the box of Section 2.5 and summed with Owen's T.

The whole profile. IRV needs the first choices among every set of remaining candidates. In a benchmark with 8 candidates it visited 106 of the 247 sets of two or more, so computing sets lazily saves little, and IRV uses the full arrangement.

All three are sums of edge terms: the Green integrals of Section 2.3 for Beta voters and the signed triangles of Section 2.5 for normal voters.

7.2. Dragging: an edge cache

An edge term depends only on the edge and the voter model. The terms are therefore cached with the endpoints as key (rounded to 10^{-9} , the smaller endpoint first, the sign from the orientation, as in Section 2.2), up to 64 MB, least recently used out first. Dragging c_k moves only the $C-1$ bisectors through c_k ; the half-planes and Voronoi edges of all other pairs keep their endpoints and come from the cache. For pairwise shares 4 of 10 half-planes are recomputed with 5 candidates, 7 of 28 with 8. The arrangement gains less: every other bisector is cut by the moving ones, so for 5 candidates about 60 of its 89 edges are new after each step.

7.3. Tabulated Beta CDF (beta_tables.py)

An edge integral evaluates G_j at Q points for every pair of nodes, $QN^2 \approx 58\,000$ calls of `betainc` at about 400 ns each. The N Beta distributions of the nodes depend only on D and the spread rule, so their CDFs can be tabulated once.

The CDF is tabulated against $s = \text{logit } y$, with the exact slope

$$\frac{dG}{ds} = g(y) y(1 - y). \quad (46)$$

Near a wall $G(y) = y^a / (aB(a, b))(1 + O(y)) = e^{as} / (aB(a, b))(1 + O(y))$: an exponential in s , smooth even for $a, b < 1$, where the density itself is infinite. A cubic Hermite interpolant with the exact slope on a uniform grid ($s \in [-40, 40]$, step 0.01) is therefore accurate everywhere ($4 \cdot 10^{-9}$ for the narrowest voters). Below the table ($y < 4 \cdot 10^{-18}$) the power law $y^a / (aB(a, b))$ itself is used, which is exact to double precision there; above it there is no double $y < 1$. (A table of logit G instead is smooth enough for a step of 0.02, but evaluating it costs an exp per point and node, which was most of the time of the compiled kernel.) Two numerical details matter:

- Of G and $1 - G$ the smaller is computed directly and the other as its complement. Using $1 - G(y) = G'(1 - y)$ with the mirrored Beta is not enough: $1 - y$ rounds to 1 below $y \approx 10^{-16}$, where $G \approx y^a$ is still 0.02 for $a = 0.1$.
- The same grid in s serves all nodes, so a point's interval and its position in it are computed once and reused for all N nodes. The remaining work per point and node is one cubic, with the coefficients of all nodes in one interval stored together.

The quantile F^{-1} has a table in logit-logit coordinates, logit x against logit u , for $u \in [\text{expit}(-30), \text{expit}(30)]$ with step 0.02. It is asymptotically linear at both ends ($x \approx (aBu)^{1/a}$ near 0), so outside the table it is extended linearly. A u -interval of width w changes an edge integral by at most w , because the integrand is bounded by 1, so values below 10^{-13} do not matter.

Over all offered deviations and spread rules the tables agree with `betainc` to $4 \cdot 10^{-9}$ and with `betaincinv` to $2 \cdot 10^{-8}$ in x (tests: 10^{-8} and 10^{-7}), and edge integrals agree with the exact ones to $5 \cdot 10^{-9}$. With the compiled kernel of Section 7.4 an edge takes about 0.2 ms instead of 23 ms. A table is built in about 0.2 s, in parallel over the nodes.

7.4. Compiled kernels (numba)

With the tables in `numpy` an edge still took 1.3 ms: about 13 passes over temporary arrays of QN^2 elements, and threads ran edges only 1.5 times faster than one, since `numpy` holds the interpreter lock between its many small operations. The loops that remain are therefore compiled with `numba`:

- the edge integrals of Section 2.3 on the tables, every form of the edge (vertical, horizontal, split, u and v) in one kernel for a whole batch of edges; the lookup, the cubic and the quadrature sum happen in one pass (about 2 ns per point and node);
- the triangles of normal voters with Owen's T (Section 2.5);
- the clipping of a polygon by a bisector (`ranking_cells`), which with a vectorized test of which cells a bisector crosses takes the arrangement of 8 candidates from 120 ms to 5 ms;
- the rounds of IRV (Chapter 3).

The kernels release the interpreter lock but are not parallel themselves; `threads.py` runs chunks of their work (edges, pixels) in a pool of threads. A kernel with `numba`'s own parallel loops would abort, with `numba`'s default threading layer, as soon as two threads call it at once, as two requests of the web UI can. Two details decided the speed: indexing the coefficient table element by element (an array view per point and node costs more than the cubic), and not calling `exp` in the innermost loop (this `numba` has no vectorized `exp`), hence the table of G itself. The first run compiles the kernels; `numba` caches them on disk, and the UI loads them at start.

Compute per drag update (in process, best of several runs, $G = 160$), with the kernels and before them:

	candidates	FPTP	Borda	Schulze	IRV
Beta, $D = 0.2$	5	5 (12) ms	11 (23) ms	15 (26) ms	14 (170) ms
	8	7 (27) ms	22 (41) ms	30 (61) ms	79 (984) ms
Beta, $D = 0.05$	5	7 (28) ms	17 (48) ms	18 (48) ms	24 (366) ms
	8	10 (52) ms	31 (95) ms	39 (96) ms	162 (2308) ms
normal, $D = 0.2$	5	6 ms	10 ms	14 ms	31 ms
	8	9 ms	23 ms	27 ms	118 ms

7.5. Borders as zero sets (methods.py, regions.py)

The shares are smooth in the median m ; only the winner jumps. Each method compares continuous functions of the shares, so its borders are where such a comparison is a tie. The *_margin variants of the methods return, next to the winner, a **margin** $\mu \geq 0$ that is continuous in the shares and 0 on every border between two winners. It is the smallest gap in a comparison that could change the winner, so it can also vanish where such a comparison ties but the winner stays; that makes no border, because the sign of ψ_c below changes only with the winner:

method	margin μ
FPTP, Borda	lead of the top score over the second
Condorcet winner	$\min_{j \neq w} (\pi_{wj} - \pi_{jw})$; in a cycle $-\max_i \min_{j \neq i} (\pi_{ij} - \pi_{ji})$
Schulze	$\min_{e \neq w} \max_f (p_{fe} - p_{ef})$
IRV	smallest gap between the two lowest tallies over all rounds

IRV. On each side of a curve where some round's two lowest tallies tie, the gap of that round tends to 0. So μ is continuous, and it vanishes on every curve where an elimination changes, whether or not the winner changes there.

Schulze. With complete rankings $\pi_{ij} + \pi_{ji} = 1$, so the margin $\pi_{ij} - \pi_{ji} = 2\pi_{ij} - 1$ orders the links exactly like the winning votes π_{ij} of Chapter 3, and the winners are the same. Unlike winning votes, the link strengths $\max(\pi_{ij} - \pi_{ji}, 0)$ are continuous in π , and so are the path strengths p . The winner w is the candidate no one beats ($p_{ew} \leq p_{we}$ for all e). It changes only where another candidate becomes unbeaten, hence the margin in the table. The simpler $\min_e (p_{we} - p_{ew})$ does not work: it can be 0 on a whole area, where the widest paths from w to e and back share their weakest link. Where a Condorcet winner exists it is the Schulze winner. There the Condorcet margin stands in for the Schulze margin: it is positive, at most the Schulze margin, and 0 on the border of the Condorcet region. So the widest paths are only computed at the points without a Condorcet winner.

For each winner c let

$$\psi_c = \begin{cases} \mu & \text{where } c \text{ wins} \\ -\mu & \text{elsewhere} \end{cases}. \quad (47)$$

ψ_c is continuous, and the region of c is $\{\psi_c > 0\}$. It is traced by marching squares (contourpy) on a grid of $G \times G$ points: the centres $(k + \frac{1}{2})/G$ plus both walls, where the medians are clamped to the outermost pixel centres, as the nodes are. On a grid edge from a point where c wins with margin α to a point where c' wins with margin β , both ψ_c and $\psi_{c'}$ cross zero at $\alpha/(\alpha + \beta)$ of the way. So neighbouring regions share their border points and leave no gaps. The shares come from the interpolant of Section 2.4, so the grid only has to be fine enough not to miss slivers; its borders do not have the steps of a pixel image. The UI uses $G = 160$ while dragging and $G = 320$ once the candidate is dropped. The margins at grid points next to a change of winner are $O(1/G)$, which the tests check.

Polygons. Every region is a list of polygons, because regions can be in pieces (Section 6.5) and can have holes (an island of another winner). Each polygon is an outer ring and its holes. Can the order of a ring's points tell the region from its complement? In principle yes: with the convention that the region lies to the left of each edge, a

counter-clockwise triangle is the triangle and a clockwise one is everything else. GeoJSON and the nonzero fill rule use this convention for holes. But a canvas never fills a lone clockwise ring as its outside; it fills the inside with winding number -1 . So the complement is sent explicitly, as an outer ring (counter-clockwise) with the triangle as a hole (clockwise). The UI fills with the even-odd rule, which does not depend on the orientation at all. This matters because flipping y for the canvas reverses every orientation. Voronoi diagrams need no grid: their cells are the polygons of Section 7.1.

7.6. Timings

Round trip of one update over HTTP while dragging one candidate ($G = 160$; median of 15 steps, $D = 0.2$, rms), which adds a few milliseconds to the compute times of Section 7.4. Timings on the test machine, a laptop with performance and efficiency cores, vary by up to a factor of three between runs.

	candidates	FPTP	Borda	Condorcet	Schulze	IRV
Beta	5	13 ms	14 ms	16 ms	16 ms	17 ms
	8	10 ms	21 ms	29 ms	26 ms	75 ms

IRV is still the slowest: it needs every cell of the arrangement, a drag changes about two thirds of their edges, and the shares of about 300 rankings are interpolated to the grid. All edges of the arrangement lie on the $C(C - 1)/2$ bisectors, though. If the integral of ω along each bisector were tabulated once, as a function of the position on it, every edge would be a difference of two values, and the cost would grow with the number of lines (28 for 8 candidates) instead of edges (468). This is not implemented.